

The Infinite Context Engine: A Self-Hosted Memory Middleware with Intent-Driven Retrieval and a Temporally-Versioned Knowledge Graph

Anonymous authors

Paper under double-blind review

Abstract

Conversational AI systems suffer from a fundamental limitation: every new session begins with complete amnesia. The user re-explains who they are, what they are building, and what decisions have already been made. The Infinite Context Engine (ICE) is a self-hosted memory middleware that sits between the user and any OpenAI-compatible language model, maintaining persistent, structured, temporally-aware memory across arbitrarily long interaction histories. ICE classifies every user prompt, retrieves relevant context from four distinct memory stores (episodic, a controlled-vocabulary semantic knowledge graph, procedural, and document), fuses the results with weighted Reciprocal Rank Fusion, and injects precisely the right context into every LLM call. We evaluate ICE using the Longitudinal State-Replay Evaluation Protocol (LSREP), a novel benchmark that replays months of conversational history through the system and measures memory quality over time. On a mature deployment across four long-form conversations (251–1,119 turns, 1,057 probes), ICE matches a vector-RAG baseline on answer quality (4.26 vs. 4.25) while injecting 32% fewer fragments and winning 30.6% of head-to-head comparisons (vs. 21.2%). On a technical-planning conversation where individual turns exceed 8,000 tokens, the vector baseline collapses to a 94.2% failure rate due to context overflow, while ICE maintains a mean score of 4.33. A feature-buildup ablation identifies Reciprocal Rank Fusion as the single most impactful component (+0.84 score delta) and reveals that several intuitively helpful features—MERA enumeration, HyDE query rewriting, dynamic token budgeting—are neutral or negative in creative domains. ICE demonstrates that less-but-better context outperforms more-but-noisier context in long-horizon conversational AI.

1 Introduction

A language model holds a conversation only for as long as its context window holds the conversation; everything older is silently gone, and the interface gives no warning that it happened. We call the resulting failure *context collapse*. Its clearest form, for us, occurs during creative or narrative work and in long technical discussions. The model usually retains the general idea of what is going on, but forgets specificity: the exact rule that was set, a particular character, a precise decision made yesterday, the FastAPI dependency-injection pattern established three months ago. The conversation continues, but the model has quietly lost the thread. Worse, the workarounds—exporting chats, copying and pasting into a fresh context, and re-explaining everything for the third time—work for the first few exchanges of a new session and then degrade, because the AI that started coherent is, by turn fifteen, a different AI that has lost the new context too. Every conversational AI interface in existence today suffers from the same fundamental flaw: amnesia. Each new session begins with complete cognitive silence.

This is not a minor inconvenience. For a human doing serious work—building a complex software system over months, constructing an original fictional universe across hundreds of sessions, planning a multi-year academic trajectory—the lack of persistent, intelligent, user-controlled memory is the single largest bottleneck between the human and the AI working as genuine collaborators. The systems designed to address this—CLAUDE.md,

`.cursorrules`, the memory layers in cloud APIs—either cap at a few hundred lines, go stale, or exist in someone else’s cloud with no user control over what is remembered and what is forgotten.

The Infinite Context Engine (ICE) is a local-first, personal AI memory middleware for human-AI conversational interfaces. It sits between the user and their language models—classifying every prompt, deciding which memories to surface, routing to the right model, managing the accumulation of knowledge over time, and injecting precisely the right context into every LLM call. ICE is not a coding agent. It is not a background tool that watches a developer’s file system. It is infrastructure for the thinking layer—for the deep collaborative sessions where the AI needs to remember that two entities are the same entity across different eras, that the user’s PostgreSQL schema is partitioned a specific way, or that the user made a particular architectural decision last Tuesday and has already ruled out the alternatives. It is memory for the human mind, not memory for a code compiler.

A crucial architectural assumption underlying ICE is that the language model itself should be treated as a stateless reasoning engine. The model has no memory of past interactions, nor does it need to. ICE does not attempt to embed memory into the model’s weights through fine-tuning or continual learning; instead, it externalises memory entirely, storing, maintaining, and retrieving context as an independent system that surrounds the model. The model processes whatever context ICE supplies and forgets it immediately after generation. This separation is deliberate: it allows models to be swapped, upgraded, or replaced without losing memory, and it makes the memory system transparent, inspectable, and user-controllable. The vector-RAG baseline shares the superficial property of feeding external context to a stateless model, but it does not reconstruct a structured, evolving cognitive state. It stores raw chunks; it does not build a knowledge graph, infer behavioural patterns, or maintain memory over time through decay and reinforcement.

In this paper we make four contributions:

1. The **ICE architecture**—a multi-store memory system with intent-driven retrieval routing, a temporally-versioned knowledge graph, access-weighted decay mechanics, and a dynamic per-query token budget.
2. **LSREP** (Longitudinal State-Replay Evaluation Protocol)—a benchmark motivated by the failure of static precision-based metrics, in which months of conversational history are replayed through the system and memory quality is measured at multiple temporal checkpoints.
3. **Empirical results** across four long-form conversations showing ICE matches vector-RAG quality on answer correctness while injecting 32% fewer fragments and winning a substantially larger share of head-to-head comparisons. On a context-overflow stress test where the vector baseline collapses to a 94.2% failure rate, ICE maintains a mean score of 4.33.
4. A **feature-buildup ablation** identifying Reciprocal Rank Fusion as the single most impactful architectural component (+0.84 score delta) and revealing that several intuitively helpful features—MERA, HyDE, dynamic budget, cluster restriction—are neutral or negative on creative-writing data.

The paper is organised as a journey of discovery rather than a victory lap. The evaluation pipeline that we eventually called LSREP was itself born from failure: an earlier precision-based metric could not capture the fact that a retrieved fragment can be “irrelevant” at turn 100 yet become critical at turn 800. Section 5 tells that story. The first full evaluation (Experiment 1) then revealed that the immature system, honestly measured, was worse than the vector baseline on every metric—a sobering result that drove the design of the mature system. The mature system, presented in Experiment 2 and the ablation, is what this paper is actually about.

2 Related Work

We organise prior work into four areas, each of which has shaped our design decisions, and we are deliberately critical of the gaps that ICE attempts to address.

2.1 Stateful Memory and Retrieval-Augmented Architectures

The architectural constraint of bounded context lengths in transformer-based models has driven the engineering of specialised memory hierarchies designed to preserve user-specific facts over extended temporal horizons. **MemGPT** (Packer et al., 2023) conceptualises the LLM context window as a form of primary random-access memory, backed by an unbounded external storage system, employing an intelligent paging mechanism to manage the control flow of data between the main context and external tiers, effectively creating the illusion of an infinite context window. While MemGPT demonstrates high efficacy in short-to-medium length interactions and intensive document analysis tasks, empirical stress tests reveal significant degradation in highly extended, multi-session dialogues. The system relies heavily on flat, dense vector retrieval, which struggles with the “compaction continuity problem” and fails to distinguish between semantically identical but contextually distinct entities over time. Crucially, MemGPT lacks a structured knowledge graph to track the temporal evolution of entities and operates without intent-driven retrieval routing, leading to context dilution in complex narrative environments.

To address the latency and token-cost bottlenecks associated with hierarchical paging, **mem0** (Chhikara et al., 2025) proposed a lightweight, production-ready memory layer that optimises the memory lifecycle by employing a single-pass, append-only extraction pipeline that distils long-form conversations into atomic, non-hierarchical facts. By utilising a multi-signal retrieval pipeline that fuses semantic embedding, BM25 keyword matching, and entity linking in parallel, mem0 achieves significant reductions in computational overhead, excelling on benchmarks such as LoCoMo and LongMemEval. Despite its efficiency, the reliance on topologically flat, key-value fact extraction inherently limits the system’s cognitive depth. It lacks the capacity for versioned entities, meaning contradictory or evolving facts are not systematically reconciled, and it does not natively support procedural memory or implement a mathematically grounded decay and archival mechanism.

MemoryBank (Zhong et al., 2024) advanced the cognitive plausibility of long-term agentic memory with an architecture heavily inspired by human psychology. MemoryBank integrates the Ebbinghaus Forgetting Curve to model memory updates via a time-aware decay and reinforcement mechanism, where every memory item maintains a continuously updated strength score. Applied to the SiliconFriend conversational agent, MemoryBank successfully maintains persona invariance and heightened empathy in multi-day dialogues. However, the system is fundamentally constrained by its retrieval substrate: it lacks a foundational knowledge graph to facilitate associative, multi-hop reasoning, and it fails to implement conversation-scoped clustering, which is essential for maintaining narrative coherence across disparate conversational threads.

SCMoE (Shi et al., 2024) developed the Self-Contrast Mixture-of-Experts framework to address the computational bottlenecks of dense memory retrieval. SCMoE introduces a training-free strategy that utilises unchosen experts in a self-contrast manner during inference, determining next-token probabilities by contrasting the output logits from strong and weak activations. While SCMoE optimises the internal routing of neural representations for better information processing, its application as an externalised memory retrieval system remains undertested, and it has not been stress-tested on the large, dynamically shifting per-turn documents that characterise persistent agentic memory.

The structural vulnerabilities of these systems underscore a pervasive industry trend: the conflation of static fact-storage with dynamic working memory. ICE addresses these precise vulnerabilities by bridging the gap between efficient retrieval and temporal narrative continuity.

2.2 Knowledge Graphs as Dialogue Substrates

To overcome the relational blindness of topologically flat vector stores, the literature has increasingly explored knowledge graphs (KGs) to map the intricate semantic topologies of textual data. **GraphRAG** (Edge et al., 2024) introduced a method designed to answer global, sense-making queries over extensive, private document collections. Traditional vector-based retrieval fails on query-focused summarisation tasks that require traversing an entire corpus. GraphRAG circumvents this by utilising an LLM to extract a massive entity knowledge graph from raw text, followed by the application of community detection algorithms (e.g., Leiden) to partition the graph. It then pre-generates hierarchical community summaries for groups of closely

related entities. While GraphRAG significantly improves answer depth, it operates under the assumption of a temporally static corpus. In GraphRAG, all ingested facts are treated as equally current; it lacks the temporal versioning required to track an entity whose properties, relationships, or states mutate dynamically across multiple conversational sessions.

KGP (Wang et al., 2023) proposed Knowledge Graph Prompting to address the specific challenges of multi-document question answering. KGP constructs a specialised graph where nodes represent individual passages or document structures, and edges denote semantic or lexical similarities. The framework deploys an LLM-based graph traversal agent that acts as a local navigator, dynamically gathering supporting context to regulate the transitional space among interconnected documents. Concurrently, **Think-on-Graph (ToG)** (Sun et al., 2024) developed a tight-coupling algorithmic framework that treats the LLM as an active agent performing iterative beam search across KG paths, prompting the LLM to explore multiple possible reasoning paths dynamically and pruning irrelevant branches until it determines that a question can be answered.

While both KGP and ToG demonstrate the profound utility of structural graph retrieval, their architectures are heavily tailored for single-shot, objective question answering over static knowledge bases. They inherently assume the graph is a ground-truth repository of factual world knowledge, completely missing the mechanisms for narrative continuity, belief decay, and contradiction resolution required for an evolving conversational agent. ICE leverages the topological benefits of these graph systems but repurposes them to support a continuously mutating conversational landscape.

2.3 Evolutions in Classic Retrieval-Augmented Generation

The foundational models of retrieval-augmented generation established the paradigm of augmenting parametric model knowledge with non-parametric, external databases to mitigate hallucinations. **RAG** (Lewis et al., 2020) formalised the original framework, providing a general-purpose fine-tuning recipe that combines a pre-trained sequence-to-sequence model with a dense vector index of a static document corpus, such as Wikipedia. While this fundamentally shifted how models access long-tailed knowledge, classic RAG relies on point-in-time document retrieval driven by maximum inner product search. Conversational history, by contrast, is not a static corpus. It grows organically, facts are frequently updated or contradicted, and context shifts dynamically. The original RAG architecture possesses no mechanisms to handle these temporal dynamics.

Subsequent iterations sought to optimise the retrieval, encoding, and decoding pipelines. **REALM** (Guu et al., 2020) integrated a latent knowledge retriever directly into the unsupervised pre-training phase, while **Fusion-in-Decoder (FiD)** (Izacard and Grave, 2021) processes retrieved passages independently in the encoder before concatenating them for joint attention in the decoder, allowing the model to dynamically weigh and aggregate evidence from up to a hundred retrieved passages. **REPLUG** (Shi et al., 2024) treats the generative LLM as an entirely frozen black box, optimising the retrieval model using perplexity scoring signals. **CRAG** (Yan et al., 2024) applies an explicit corrective process to mitigate retrieval noise, implementing a lightweight evaluator that scores retrieved documents and triggers a decompose-then-recompose algorithm to filter irrelevant spans.

Despite their mathematical and architectural elegance, REALM, FiD, REPLUG, and CRAG are structurally confined to static knowledge bases. They possess no innate mechanisms to handle a personalised, highly dynamic, and evolving conversation history where a user’s preferences, constraints, and state of mind shift iteratively. ICE replaces the static-corpus assumption with a highly mutable, chronologically aware middleware designed specifically for personal agentic history.

2.4 Algorithmic Primitives in ICE

To achieve resilient memory retrieval, ICE synthesises several foundational algorithmic techniques. **BM25** (Robertson and Zaragoza, 2009) remains one of the most successful sparse retrieval algorithms, scoring document relevance based on a non-linear term frequency function and inverse document frequency, with document length normalisation. While dense semantic embeddings excel at capturing conceptual similarity, they frequently suffer from catastrophic failures in exact-keyword matching. BM25 ensures high-fidelity

lexical recall and serves as a secondary leg in ICE’s multi-signal retrieval strategy. **Reciprocal Rank Fusion** (Cormack et al., 2009) is the mathematically elegant algorithm that combines document rankings by sorting them based on the inverse of their rank positions across multiple independent search systems, preventing any single retrieval methodology from disproportionately skewing the results. RRF is ICE’s indispensable fusion backbone, and our own ablation confirms that it is the single most critical mathematical component for retrieval stability. **HyDE** (Gao et al., 2023) addresses the vocabulary mismatch problem by generating a hypothetical document from the query and embedding that instead, but our ablation shows it to be largely neutral in creative domains, indicating that its utility must be dynamically gated.

2.5 The Gap

The gap in prior work is not any single missing feature—it is the combination of temporal awareness, structured entity memory, intent-driven retrieval routing, and a per-query token budget. Systems that add one of these still fail on the specific use case that motivated ICE: a long-running creative or technical project in which entities evolve, intentions shift, and conversation history must be filtered as much as retrieved.

3 Datasets

The evaluation uses four long-form conversational datasets, all authored by a single user over periods ranging from months to years. We use anonymised descriptive labels rather than the internal names. table 1 summarises them.

Table 1: Four evaluation datasets (anonymised). All are single-user, self-authored. The “Stress dimension” column indicates which aspect of the ICE architecture each dataset most heavily exercises.

Label	Description	Turns	Ckpts.	Stress dimension
Dataset A (Creative Writing)	Collaborative fan-fiction writing session	290	10	Narrative continuity, character evolution
Dataset B (Long-Form Creative)	Fantasy world-building and story-planning	1,119	20	Temporal-horizon retrieval, entity evolution
Dataset C (Technical Planning)	Design and development of ICE itself	325	8	Information density, 8K+ token turns
Dataset D (Academic Planning)	Graduate school and career planning	251	12	Mixed personal/technical, decision tracking
Total		1,985	50	

Each dataset was selected for three reasons. First, all four are substantially longer than typical chat interactions (251–1,119 turns), providing opportunities for information to become separated by hundreds of turns and creating realistic long-term retrieval challenges. Second, each represents a distinct memory domain—creative-writing datasets emphasise narrative continuity and entity consistency, the technical-planning conversation emphasises design decisions and implementation rationale, and the academic-planning conversation requires tracking personal preferences, goals, and decision histories. Third, all four contain dense memory structures—recurring entities, evolving relationships, technical terminology, procedural patterns, and long-term dependencies—suitable for exercising ICE’s multi-retrieval architecture.

We acknowledge the single-user limitation: these conversations are genuinely long and genuinely demanding in ways that synthetic benchmarks are not, but they do not yet represent a diverse population of users.

4 System Architecture

ICE is an OpenAI-compatible memory middleware that sits between a conversational client and a pool of locally-served large language models (Ollama / SGLang). It is not a model itself: every request addressed to

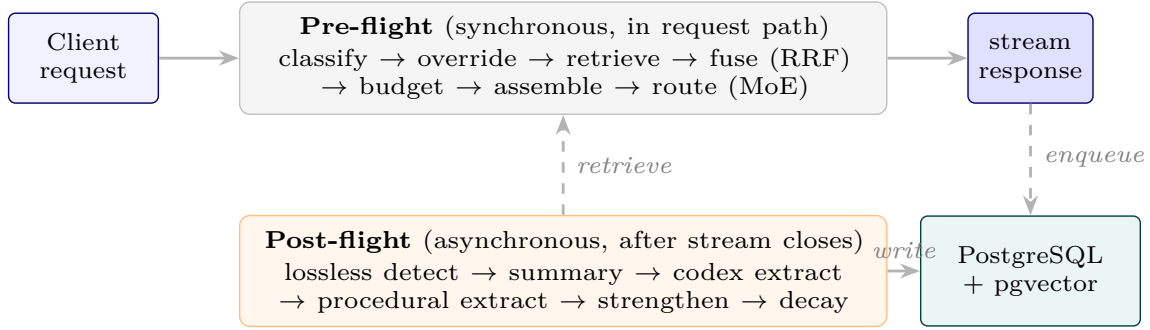


Figure 1: ICE system overview. The pre-flight phase runs synchronously in the request path; the post-flight phase runs asynchronously after the stream closes.

the synthetic model name `ice-proxy` is intercepted by a FastAPI proxy that classifies the turn, retrieves relevant context from four long-lived memory stores, assembles a KV-cache-friendly prompt, routes the request to a per-turn specialist model, streams the response back over Server-Sent Events (SSE), and then dispatches a fan-out of background workers that extract, decay, cluster, and crystallise the new turn into long-term memory. The net effect is that any downstream model operates against an effectively unbounded, personalised context that is selectively rebuilt on every turn.

Each turn traverses a **pre-flight** (synchronous, in the request path) and a **post-flight** (asynchronous, after the stream closes) phase. In the pre-flight phase the user message is classified by a two-stage pipeline—a rule-based Dynamic Intent Inferencer (DI3) followed, on miss, by a 25-way PyTorch MLP—producing topic tags, intent tags, and a context-reliance label. Hard override rules then coerce the label (e.g. `Creative & Media` $\Rightarrow$ `Long Term Memory`). A Hybrid Retrieval Orchestrator runs six retrieval legs (BM25, vector with decay weighting, Codex graph traversal, procedural, RAG, batch summaries), fuses them with weighted Reciprocal Rank Fusion (RRF), and post-processes the fused list with bonuses, session diversification, deduplication, and a dynamic token budget. A Prompt Assembler concatenates the retrieved fragments with persistent memory slots, recent turns, and the live user message under a stable prefix that maximises KV-cache reuse. Finally, a Mixture-of-Experts (MoE) router selects the best locally-served model for the assembled prompt, with per-conversation stickiness. In the post-flight phase a Post-Flight Evaluator runs lossless detection and summary generation, unconditionally dispatches a Procedural Extractor and, conditionally on the lossless flag, a Codex Extractor; both are idempotent Celery tasks. A fan-out of scheduled background workers—decay, clustering, reflection, batch summariser, sentinel monitor, and a weekly fine-tune worker—maintains the memory stores over time.

4.1 Classification Engine

The classification engine produces, for each user turn, a triple of outputs: a set of topic tags, a set of intent tags, and a single context-reliance label. These three outputs drive every downstream decision—which retrieval legs are weighted, how the token budget is split, whether the wide-net fallback fires, and which model the MoE router selects. The engine is deliberately a two-stage cascade: a cheap rule-based pre-classifier resolves obvious cases in microseconds, and a small PyTorch MLP resolves everything else with sub-millisecond CPU inference.

The MLP is a deliberately tiny classifier (a $384 \rightarrow 128 \rightarrow 25$ multi-layer perceptron with ReLU and 0.3 dropout). The input is a single 384-dimensional embedding produced by a frozen `Qwen/Qwen3-Embedding-0.6B` embedder; the single shared linear head is sliced at inference time into three blocks: 11 topic labels, 11 intent labels, and 3 context-reliance classes (`Zero Shot`, `Long Term Memory`, `Real Time Search`). Topic and intent are decoded multi-label (sigmoid, threshold 0.3, argmax fallback); context-reliance is decoded single-label (softmax, argmax).

DI3 (gated by a settings flag) is a rule-based pre-classifier that catches obvious cases before the MLP runs. It computes five density signals in $[0, 1]$ from the raw prompt: code density (token-weighted sum over code features), sentiment density (lexicon match), meta density (references to the model itself), noise density

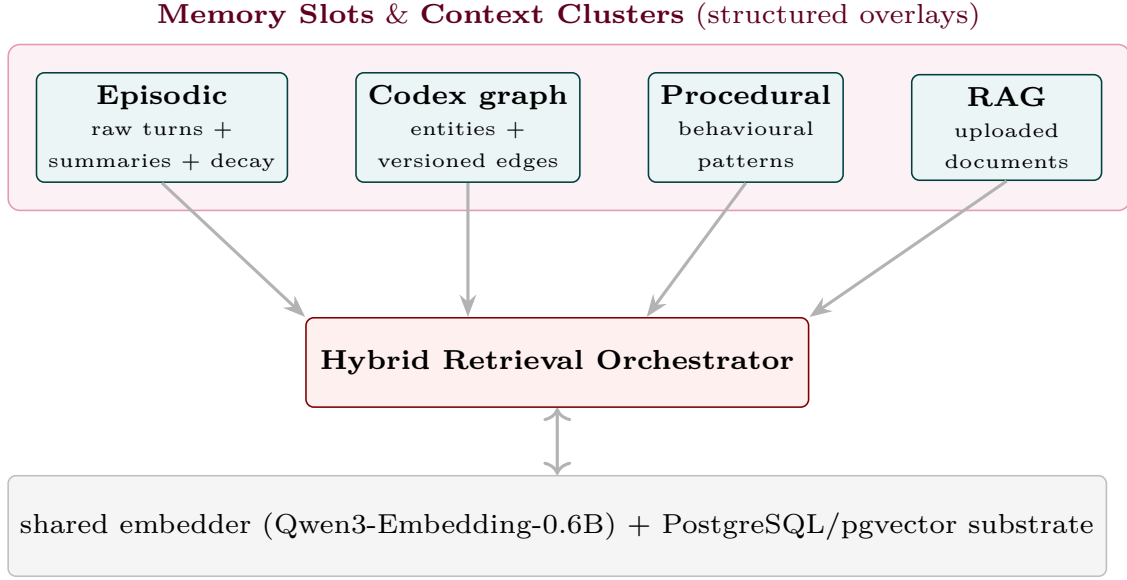


Figure 2: Memory architecture. The four stores differ in cardinality, schema, and retrieval semantics, but share a common embedder and a common PostgreSQL substrate.

(short, alpha-poor, keyboard-mash), and reference density (anaphoric terms like “this”, “that”, “it”). DI3 evaluates five rules in order, returning the first that fires, or none, in which case the MLP runs. The reference rule is special: it returns empty topic and intent tags but forces Long_Term_Memory, and the orchestrator then runs the MLP only to obtain the topic and intent labels. A two-tier threshold on the reference rule (0.2 for short conversations, 0.1 once the conversation exceeds ten turns) is the engine’s primary mechanism for implementing a long-conversation LTM bias.

Override rules. After DI3 or the MLP produces a result, three rules run in evaluation order: (i) LTM immutability—once Long_Term_Memory is set, no override may downgrade it; (ii) Creative_& Media $\Rightarrow$ LTM; (iii) Software_& Tech + anaphora $\Rightarrow$ LTM (substring check on 23 referential words including “my”, “our”, “this”, “previous”). A second, API-level LTM bias lives outside the classifier: when a conversation has more than ten turns or the classifier’s max_confidence < 0.95, a Zero_Shot context-reliance label is upgraded to Long_Term_Memory before retrieval runs.

Context-aware classification. When a conversation ID is supplied, the MLP path queries the last three episodic-memory turns for that conversation (n=3, max_total_words=500), preferring summary_text and falling back to the first 150 words of raw_text. The context is prepended to the prompt as natural-language text before embedding.

4.2 Memory Architecture

ICE maintains **four distinct memory stores**, plus Memory Slots (persistent structured working memory), Context Clusters (conversation-scoped topical clusters), and a cold-storage archive. All stores live in a single PostgreSQL database with the pgvector extension; every vector column is **Vector(384)** because the same Qwen3-Embedding-0.6B embedder is used everywhere.

Memory slots are prepended verbatim to every prompt. Seven slot names are server-enforced: **persona**, **user_preferences**, **tool_guidelines**, **project_context**, **guidance**, **pending_items**, **session_patterns**. Slots are written through four paths: direct user update, batch initialisation, Reflection-proposed but user-gated updates (high-stakes slots land in the review queue), and Reflection auto-applied updates to **pending_items** only.

Context clusters are conversation-scoped topical groupings. The centroid of member turn embeddings (renormalised to unit length after every change) is a **Vector(384)**; membership is many-to-many through an

Table 2: The four memory stores. Each store is queried by one or more retrieval legs in the hybrid orchestrator.

Store	Purpose / Key columns	Retrieval method
Episodic	Every conversational turn; raw_text, summary_text, lossless_flag, decay_score, access_count, is_document, embedding	BM25 leg, vector-with-decay leg, session diversification
Codex	Temporally-versioned semantic graph; entities (canonical name, aliases, properties JSONB, context_payload), typed edges with valid_from / valid_until, append-only event log, snapshots	Graph traversal (BFS depth 3, valid_until IS NULL), MERA enumeration fallback
Procedural	Recurring behavioural patterns; pattern_name, trigger_conditions, reinforcement_count, confidence_score, is_active	Vector top-5 with hard intent gate (Strategic_Planning, Generation, Open_Exploration)
RAG	Externally-ingested documents; chunked at 512 words, embedded	Vector top-5, triple-gated (LTM AND (Factual_Retrieval $\vee$ Analysis_& Summarization) AND reference keyword)

episodic_cluster_links join table. The clustering worker (beat every 30 min) assigns each unassigned turn to the best existing cluster (combined score: embedding similarity + 0.08 per shared entity + tag overlap) or to a new cluster when no candidate clears a 0.6 similarity threshold.

Decay mechanics are access-weighted. Three independent decay workers (all beat-scheduled every 5,400 s for 16 cycles/day) apply per-cycle multipliers: $0.95^{1/16}$ ($\approx 5\%$ /day) for unaccessed, non-creative turns; $0.98^{1/16}$ ($\approx 2\%$ /day) for accessed, non-creative turns; $0.99^{1/16}$ ($\approx 1\%$ /day) for any turn tagged Creative_& Media. A **creative floor** clamps decay_score to 0.3 for creative turns regardless of age or access, protecting long-form narrative from being summarised away. Turns whose decay_score falls below 0.1 are archived; those that subsequently fall below 0.05 are moved to a minimal cold-storage table and physically deleted from episodic memory. Codex edges decay by a similar mechanism (strength multiplied by $0.99^{1/16}$; demoted from active to pending once strength < 0.3). Procedural patterns are boolean-deactivated after 180 days if they have fewer than three reinforcements.

Batch summaries group decayed-but-not-archived turns into 50-turn batches; a background model produces a 500-token preservation-prompt summary, and the summary is the second life of a turn—retrievable after the original ages into cold storage.

4.3 Codex Knowledge Graph

The Codex is a temporally-versioned, controlled-vocabulary knowledge graph spread over four tables: **codex_entities** (canonical name, aliases, properties JSONB, auto-regenerated context_payload, embedding), **codex_edges** (typed relations with strength, confidence in {pending, active}, valid_from, valid_until, where NULL means currently true and non-NULL means historically superseded), **codex_events** (append-only event log), and **codex_snapshots** (compaction output). Its central design lever is a three-bucket relation vocabulary: **property** (11 categories written to the properties JSONB; e.g. name, age, role), **multi-valued** (16 categories, e.g. uses, friend_of, cites; new edges never expire prior ones), and **single-valued** (11 categories, e.g. works_at, married_to; a new edge auto-expires the prior active edge of the same (source, relation) pair). Generic dumping-ground relations (**is**, **has**, **connects_to**) are deliberately removed from the vocabulary so the extractor is forced to be specific.

The **handle_triplet** write path is the heart of temporal versioning. Entity resolution is two-stage: exact canonical-name match, then alias match, else create with a deterministic UUIDv5. The write branch then depends on the bucket: a property observation expires prior active edges of the same (source, relation), writes a new edge, and overwrites the JSONB key; a reinforcement of an existing non-property edge increases strength and promotes from pending to active at strength ≥ 2 ; a new relation between the same pair expires the old one only if the old relation is not multi-valued. Every state change emits a CodexEvent; the entity’s context_payload is rebuilt from the property map plus the ten most recent active outgoing edges.

MERA (Meta-Enumeration Retrieval Agent) is a fallback for category-based queries (“list all the characters”, “which libraries does the project use”). It activates only when NER extracts no entities AND the prompt contains a category trigger and an enumeration hint. Candidates are deduplicated and ranked by 30-day episodic-mention count.

Micro-NER is a BIO tagger (a 3-layer MLP $384 \rightarrow 128 \rightarrow 64 \rightarrow 3$) that classifies per-token embeddings into B-ENT, I-ENT, O. When the trained model is unavailable, the system falls back to a regex `\b[A-Z][a-zA-Z]{2,}\b` minus a stoplist. **Vector fuzzy matching** for entity resolution embeds the prompt entity strings, linearly scans every entity embedding, and takes the best-scoring one above cosine 0.85; this is the mechanism that lets the Codex resolve “Obsidian Citadel” to its canonical entity even when the user spells the name slightly differently.

The Codex retrieval leg extracts prompt entities via the shared NER, resolves them by vector fuzzy matching, optionally restricts to a conversation scope, and performs a BFS traversal to depth 3 over edges where `valid_until` IS NULL, appending “[Entity: {canonical_name}]\n{context_payload}” per visited entity. The event log is compacted by a worker that snapshots any entity with more than 100 uncompact events—textbook event sourcing, with the live edge table as current state, the event log as audit trail, and the snapshot table as bounded replay.

4.4 Hybrid Retrieval Orchestrator

The orchestrator is the core of the pre-flight phase. Before any retrieval leg executes, the orchestrator receives a classification that may have been altered by two layers of LTM bias (the classifier’s hard overrides and the API-level conservative override), so retrieval runs for almost every turn in a long conversation regardless of the classifier’s initial belief.

The six retrieval legs are: **BM25** over a Postgres tsvector of `raw_text` and `summary_text`, filtered by `decay_score > 0.2` AND `is_archived = false`, LIMIT 100, ordered by `ts_rank`; **vector** with decay weighting— $(1 - (\text{embedding} \Leftrightarrow \text{prompt_embedding})) \cdot \text{COALESCE}(\text{decay_score}, 1.0)$ —which is what distinguishes this leg from pure semantic search (a high-similarity but decayed turn is down-ranked); **Codex graph traversal** with NER $\rightarrow$ vector fuzzy matching $\rightarrow$ BFS depth 3 over valid edges; **procedural** with a hard intent gate (only Strategic_Planning, Generation, Open_Exploration), vector top-5, conversation-scope and trigger-condition gates; **RAG**, triple-gated (LTM AND (Factual_Retrieval $\vee$ Analysis_& Summarization) AND reference keyword) and global, not conversation-scoped; and **batch summaries**, conversation-scoped top-3 by cosine similarity.

Dynamic leg weighting. Base weights are {bm25: 0.8, vector: 1.0, codex: 0.5, procedural: 0.2, rag: 1.0}. Five intent profiles override the four non-RAG legs. For example, Factual_Retrieval boosts vector to 1.2 and demotes codex to 0.1; Generation / Ideation / Open_Exploration boost codex to 1.2 and demote vector. Two cumulative topic overrides then apply: Creative_& Media $\Rightarrow$ codex + = 0.3; Software_& Tech $\Rightarrow$ procedural + = 0.4. Every weight is floored at 0.

Reciprocal Rank Fusion combines the legs:

$$\text{score_RRF}(f) = \sum_{\ell \in \text{legs}} \frac{\alpha_{\ell}}{k + \text{rank}_{\ell}(f)}, \quad k = 60, \quad (1)$$

where α_{ℓ} is the blended weight (defaulting to 1.0 for unknown legs) and $\text{rank}_{\ell}(f)$ starts at 1. Fragments are deduplicated *during* fusion—the first occurrence is registered and subsequent occurrences add to its RRF score. The output is sorted by RRF score descending.

Post-fusion processing applies five sequential transforms: additive bonuses (keyword +1.0, length +1.5 if `word_count > 800` or -0.7 if < 80 , recency +1.0 if top-10% most recent, but recency is *skipped* for Creative_& Media turns because recent meta turns are noise for creative work); session diversification (max 3 fragments per foreign conversation, the active conversation is uncapped, RAG/Codex/procedural are always kept); deduplication (SHA-256 of text, first occurrence wins); a two-phase token-budget enforcement that first guarantees leg diversity (the best fragment of each leg is added first if it fits) and then greedily fills the

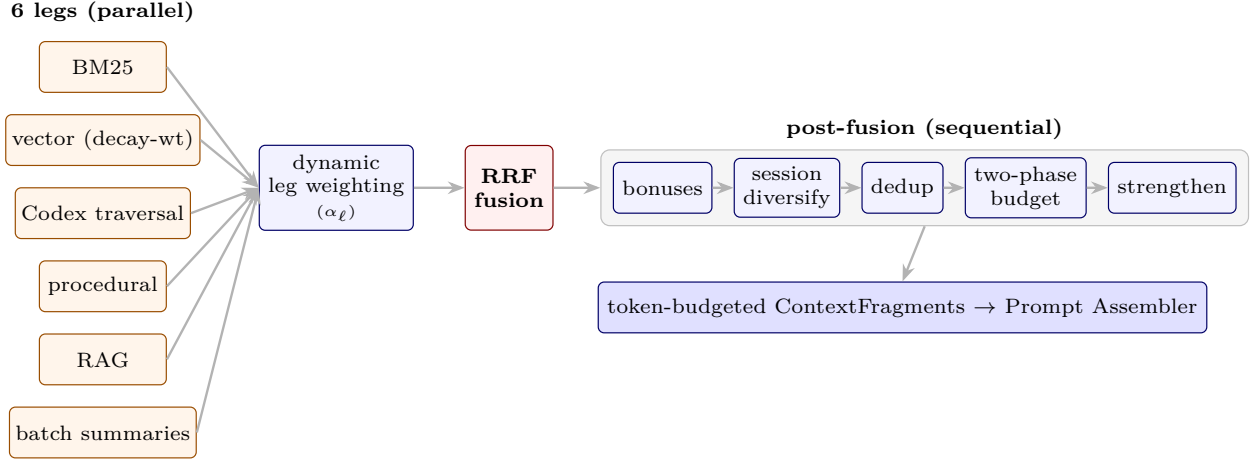


Figure 3: Retrieval orchestrator flow. The six legs are heterogeneous in their retrieval semantics; RRF is the only mechanism that allows them to be combined without score-distribution normalisation; the two-phase token budget guarantees leg diversity under budget pressure.

remainder; and finally a *strengthening* step that increments `access_count` and adds 0.15 to `decay_score`—retrieval acts as a partial reversal of decay.

Cluster-scoped retrieval identifies the most relevant clusters (top-30 by centroid cosine, re-scored as combined = similarity + 0.3 · tag overlap + 0.15 · name similarity) and restricts episodic retrieval to those cluster IDs unless the best combined score is below 0.50, in which case the orchestrator falls back to global search.

Dynamic token budget. `TOTAL_CONTEXT_BUDGET` = 23,000, `OVERHEAD_RESERVE` = 1,800, so available = 21,200. A recent-window fraction (length-based, reduced by token density, shifted by intent and topic, clamped to [0.05, 0.85]) splits this between recent turns and retrieval. A *growth cap* prevents long-tail conversations from over-allocating: <30 turns → 2,000 + 150*n*; <100 → 5,000 + 100(*n* − 30); <500 → 10,000 + 30(*n* − 100); else raw retrieval. Leftover budget is intentionally unused—this is the lever that keeps ICE token-efficient relative to a vector-only baseline. A **wide-net fallback** short-circuits to a full vector scan with no conversation or cluster filter, a 2,000-token ceiling, and single-leg RRF when the classifier’s `max_confidence` < 0.75, keeping the response focused when the system is unsure.

Feature toggling. A `ConfigurableOrchestrator` subclass exposes a dictionary of flags that toggle retrieval legs and post-processing steps on or off, without redefining classification or fusion logic. The ablation study in Section 6.4 uses this mechanism to add features cumulatively from a bare vector baseline.

4.5 Prompt Assembly

The Prompt Assembler concatenates the retrieved fragments with persistent memory slots, recent turns, and the live user message into a list of chat-completion messages. The ordering is deliberately *stable-prefix* to maximise KV-cache reuse: (i) a fixed system message with an inline `=== PERSISTENT CONTEXT ===` block rendering each active slot; (ii) recent turns (alternating user/assistant pairs, dynamic per-turn word caps); (iii) a single user message with header `=== RETRIEVED CONTEXT ===` (optionally tagged with cluster names when cluster scope is populated); (iv) a single assistant “acknowledgment” message that acts as a boundary marker so the model treats the final user message as the live question rather than another history turn; (v) the live user question. Because the system message, slots, and most of the recent-turns prefix change slowly, most of the prefix K/V tensors are reusable across consecutive requests.

A two-layer budget process enforces the final size: the orchestrator’s `_enforce_token_budget` packs fragments to `max_retrieval_tokens`, and the API layer recomputes `total_words` against a per-model ceiling (2,770 words for the deployed model) and iteratively pops procedural and episodic fragments until the budget is

met, while preserving RAG and Codex fragments because they are the hardest to recompute and the most likely to carry the answer.

The self-maintaining background worker cluster (fourteen scheduled and event-triggered workers) and the operational infrastructure (FastAPI service, PostgreSQL/pgvector substrate, Celery/Redis worker fleet, and configuration system) are described in Appendix A.1. A complete technical reference—including the full training pipeline for the classifier and NER models, the controlled relation vocabulary in its entirety, GPU resource management, and idempotency architecture—is available in the technical report archived in the repository (`docs/ICE_Architecture[real_v2].md`), which describes the system at the tagged evaluation snapshot.

5 Experimental Design

5.1 Why LSREP Was Needed: The Experiment 0 Story

Our first evaluation attempt (*Experiment 0*) used a precision-based approach: measure whether retrieved fragments contained the relevant information. We constructed probes from a corpus of 41 conversations and asked: did the orchestrator retrieve the fragment with the exact timestamp or ID mapped in the ground-truth file? This failed for two compounding reasons. The first was what we now call the *stateless pronoun problem*—held-out probes were extracted from their original conversations and tested in a vacuum, stripped of surrounding turns; a probe like “*you mentioned that the current script saves nothing*” was fed to the orchestrator with zero surrounding context, so the system pulled fragments that were mathematically correct in isolation but functionally useless. The second was the *incompetent judge*—a 7B grading model that hallucinated failures, scoring 0 for fragments a human immediately recognised as the exact brainstorming session being asked about. Manual re-scoring on a subset of the probes produced 31% precision rather than the automated 13.87%, and the gap was attributable to the judge, not the retrieval system.

The failure of the stateless precision benchmark led directly to the idea of LSREP. If a weak judge could not be trusted to score fragments, the system needed to be asked to answer real questions, and the answers themselves needed to be judged. And if memory accumulation was to be measured, the same questions needed to be asked at multiple points in time, not just once at the end. The first sketch was: replay a conversation from the beginning, pause at regular intervals, ask hand-written questions, and score the answers against the full conversation history. That sketch became LSREP.

5.2 LSREP: The General Protocol

LSREP freezes a historical conversation at an arbitrary temporal slice T_n , reconstructs the entire database and memory-lifecycle state exactly as it existed at that moment, injects evaluation probes, and then evaluates the system’s answers with an independent judge. Its defining choice is to evaluate the *model’s output*, not the retrieval pipeline’s intermediate results—because a fragment that is “irrelevant” at turn 100 may become critically relevant at turn 800, and a fragment that differs from the ground truth may still be *equally valid* (the multiple-correct-answer problem that sank Experiment 0).

The general procedure is: for each historical conversation, determine the total turn count L and select split points; wipe the target database and inject turns T_1 through T_{n-1} sequentially into the episodic memory tables, preserving exact relative or real timestamps; trigger the background workers synchronously (decay computes initial `decay_score` metrics, the codex extractor builds entity nodes and relationship edges, the procedural extractor scans for behavioural motifs, and the clustering worker aggregates related turns); then administer the probes and judge the answers.

This skeleton is common to both experiments. *However, the two experiments instantiate it very differently*—in split range, probe construction, ground-truth generation, checkpoint handling, and the evaluated system itself. Because those differences are large enough to affect how the results should be read, we describe each experiment’s environment separately below, and summarise the differences in table 3. A single caveat frames everything that follows: **Experiment 1 is a pilot**. Its role was to expose measurement flaws and immature

subsystems; the paper’s substantive evaluation is Experiment 2, and Experiment 3 is the ablation that explains *why* the mature system behaves as it does.

5.3 Experiment 1 Environment (Pilot)

System under test. Experiment 1 evaluated an immature build with only the vector and BM25 retrieval legs meaningfully active. The classifier used a frozen MiniLM (`all-MiniLM-L6-v2`) embedding backbone; retrieval used static per-leg limits and a static global token budget (5,000 tokens) plus an uncapped 10-turn sliding window; the Codex knowledge graph was effectively non-functional (regex-only NER, a 1.5B extractor, and a corroboration quorum that was rarely met); and cluster-scoped retrieval did not yet exist. Two of these—the static budget and the uncapped sliding window—directly caused the token-accounting problem described below.

Protocol instantiation. Conversations shorter than 10 turns were excluded, reducing the corpus from 42 (the Experiment 0 corpus) to 18. Split points were drawn from $[\max(10, 0.30L), 0.95L]$, three per conversation with a fixed seed; each split defined a historical block (turns 1..*s*) and a future reference block (the next 10 turns). Probes were *hand-written*, 8–30 per conversation, and probes from earlier splits were re-asked at later splits to observe change over time. Ground truth used a *single* retrieval-assisted pass: the probe was embedded, similar turns retrieved (20–40 turns, 30,000-token cap), and a language model compiled the retrieved evidence into one reference answer—frozen for that probe. Six conditions were run per probe (two control / sliding-window-only, two vector-RAG, two Full ICE, each in generalist and MoE variants).

The token-accounting audit. After Experiment 1 completed, an audit found that the reported context size for Full ICE included retrieved fragments but *omitted* the recent-turn sliding window incorporated during prompt assembly. The generated answers, scores, rankings, and hallucination labels were unaffected—the sliding window was correctly supplied to the model at inference time; only the reported context-efficiency statistics were wrong. We replayed the sliding-window assembly (10 turns, 500-word-per-turn cap) for every checkpoint and added the omitted tokens. The correction did not change the qualitative findings, but it did invalidate any claim of superior token efficiency: corrected, Full ICE consumes *more* total tokens than the vector baseline at roughly equal quality. Experiment 1 is therefore a proof-of-concept for memory quality and retrieval behaviour, not a demonstration of context compression. This finding directly motivated Experiment 2’s unified dynamic token budget. All Experiment 1 numbers reported in this paper are the corrected values.

5.4 Experiment 2 Environment (Mature System)

System under test. Experiment 2 evaluated the mature system: the classifier backbone was upgraded to Qwen3-Embedding-0.6B and the head retrained from scratch; a conservative routing override was added (favour retrieval under low confidence or long conversations); the static budget and uncapped sliding window were replaced by the unified *dynamic* token budget of Section 4; and *cluster-scoped retrieval* was introduced, restricting episodic search to the most relevant topical clusters rather than the whole conversation. These changes are the reason Experiment 2 can inject fewer fragments while holding quality: the gain comes from cleaner, more focused evidence, not more of it.

Protocol instantiation. Rather than many conversations at isolated checkpoints, Experiment 2 followed four carefully chosen long-horizon conversations (Section 3) across their entire lifespan. Three instantiation choices differ sharply from Experiment 1. (1) *Continuous replay with preserved state*: each conversation was replayed chronologically into one fresh deployment, and memory stored at an early checkpoint remained available at every later checkpoint (in Experiment 1 each checkpoint was effectively independent). A simulated memory lifecycle—decay, codex-edge decay, procedural decay, reflection, clustering, consolidation, sentinel monitoring—ran *between* checkpoints so retrieval reflected aged, consolidated memory rather than fresh storage. (2) *Automatic, retrieval-leg-aware probe generation*: probes were generated at each checkpoint from a sampled history window (first 5 turns, last 10 before the checkpoint, 15 random from between), using only information available up to that checkpoint, and were explicitly targeted at specific legs (Codex, vector, BM25, procedural). Every probe required a unique temporal anchor to prevent ambiguity across hundreds of turns. These auto-generated probes were combined with the 72 hand-written probes inherited from Experiment 1. (3) *Incremental evaluation*: every probe whose origin was at or before a checkpoint was re-evaluated at that

checkpoint, so a probe generated early is scored repeatedly as history accumulates—which is what makes longitudinal memory growth directly observable.

Evolving ground truth. The single frozen ground truth of Experiment 1 was replaced by a three-stage temporal-refinement pipeline: (i) *forensic regeneration* of the origin-checkpoint answer under a stricter, evidence-oriented prompt that records temporal provenance and distinguishes historical from current state; (ii) *temporal anchoring*, attaching to each probe an anchor identifying the specific fact/event it tracks; and (iii) *forward propagation*, updating each answer at every later checkpoint when new content contradicts, updates, or extends it, while anchor-preservation checks guard against silent drift to a superficially similar later event. The result is that each probe carries a *temporally evolving* reference document rather than a static answer—necessary because otherwise a system that correctly *updated* a fact could score worse than one that repeated an outdated fact.

Scoring differences. Experiment 2 added temporal-aware scoring (an answer can lose points for reporting a superseded fact even if it was once correct) and inference credit (a specific, evidence-consistent detail not explicitly in the reference is rewarded, not flagged as hallucination). Structural-relevance assessment credits fragments that provide supporting background without containing the answer verbatim.

Human verification, not score alteration. Two independent pieces of human involvement in Experiment 2 are worth stating precisely, because “manually verified” can be misread as “scores were adjusted by hand.” They were not; the human role was verification and correction of the judge’s own labels, not substitution of human judgment for the judge’s. First, the 72 hand-written probes inherited from Experiment 1 (Section 5.3) were re-evaluated by hand against their absolute scores and tournament rankings, so a genuine human-authored, human-checked evaluation slice sits inside the Experiment 2 benchmark alongside the automatically generated and automatically judged probes—not merely the model’s own self-consistency. Second, every hallucination flag across all 1,211 probes was manually audited. This second correction was necessary because the automated judge was systematically too harsh across *all four* conditions—generalist and MoE, ICE and vector alike—whenever an answer contained a specific, true, and directly relevant detail that was simply absent from the necessarily condensed ground-truth dossier; the judge tended to flag such details as unsupported rather than recognise them as correct supplementary information. The audit corrected these false positives wherever a detail was verifiably true against the source conversation, applying the same standard to every condition; it did not remove flags for genuine fabrications, and we deliberately did not over-correct in the other direction by crediting speculative or unverifiable additions. This audit was applied to Experiment 2 only; Experiment 3 (the ablation) uses the automated judge’s hallucination labels unmodified, for the bias reasons discussed in Section 8.

Ground-truth and judge. Both experiments use the same independent judge, `mattbucci/gemma-4-12B-AWQ` on SGLang (150,000-token context, temperature 0.0, deterministic decoding), never used for retrieval, generation, probe generation, or ground-truth construction. The extraction rules—Strict Neutrality, Verbatim Anchor Preservation, Temporal Dominance, Third-Person Attribution—are shared; what differs is that Experiment 2 applies them through the evolving-dossier pipeline above rather than a single pass.

Evaluation conditions (Experiment 2). The mature evaluation used a four-condition matrix: **Vector RAG Baseline (Generalist)**—single-leg pgvector similarity, top-K=30, no decay weighting, classification, or fusion, generalist model (`gemma4:26b-a4b-it-q4_K_M`); **Vector RAG Baseline (MoE)**—same retrieval, expert-routed; **Full ICE (Generalist)**—all six legs, RRF fusion, classification, decay weighting, dynamic budget, post-fusion processing; and **Full ICE (MoE)**—same ICE retrieval, expert-routed. The vector baseline is not a straw man: it uses the same embedding model, database, and token-budget logic, differing only in single-leg versus six-leg retrieval. The sliding-window-only control conditions from Experiment 1 were dropped, having already been characterised.

A note on conservatism. Because history is *replayed* rather than lived, the reinforcement that normally strengthens frequently-referenced memories during live use is largely absent—the primary reinforcement signal becomes the evaluation probes themselves. A concept referenced dozens of times in real use and one mentioned once therefore look more alike under replay than they would in deployment. Experiment 2’s numbers should thus be read as a conservative lower bound on deployed performance, not a ceiling (see Section 8).

Table 3: What differs between the two LSREP instantiations. Experiment 1 is a pilot; Experiment 2 is the paper’s substantive evaluation. The shared skeleton (state reconstruction, synchronous worker trigger, output-level judging by the same independent judge) is described in Section 5.2.

Dimension	Experiment 1 (Pilot)	Experiment 2 (Mature)
System maturity	Vector + BM25 legs; MiniLM classifier; static budget; uncapped sliding window; Codex non-functional	Six legs; Qwen3 classifier + routing override; dynamic budget; cluster-scoped retrieval
Corpus	18 conversations (isolated checkpoints)	4 long-horizon conversations (full-lifespan replay)
Split range	$[\max(10, 0.30L), 0.95L]$, 3 splits/conv	Length-adaptive checkpoints (8–20), even spacing with jitter
Probes	Hand-written (8–30/conv)	Auto-generated + leg-aware (147) plus 72 inherited hand-written
Ground truth	Single retrieval-assisted pass, <i>frozen</i>	Three-stage forensic regeneration + forward propagation, <i>evolving</i>
Checkpoint state	Independent per checkpoint	Preserved across checkpoints; lifecycle runs between them
Probe evaluation	Each probe once	Incremental: re-scored at every later checkpoint
Scoring	Static correctness	Temporal-aware + inference credit; hallucination flags audited for false positives
Conditions	Six (incl. sliding-window controls)	Four (controls dropped)

5.5 Metrics

Table 4: Evaluation metrics. Each metric is computed per probe and aggregated across the benchmark.

Metric	Definition	What it measures
Score (1–5)	Mean absolute score from the judge	Answer quality
SPF (Score per Fragment)	$\text{mean_score}/\text{mean_fragments}$	Retrieval efficiency
TUR (Token Utilisation Rate)	$\text{mean_score}/(\text{mean_tokens}/1000)$	Token efficiency
Win Rate (%)	Blind tournament ranking: all four conditions’ answers for a probe are anonymised, shuffled, and ranked best-to-worst by the judge in a single pass, independent of the absolute (1–5) scoring; win rate is the share of tournament appearances placed first	Relative preference, judge-order-bias-free
Hallucination (%)	Percentage of probes where the judge detects fabricated information not present in the conversation	Faithfulness
Fragment Count	Mean fragments injected per probe	Retrieval volume
Recency Delta	$\text{mean_score}(\text{late bins}) - \text{mean_score}(\text{early bins})$	Longitudinal improvement

Where confidence intervals are reported, they are 95% percentile-bootstrap intervals computed by resampling probes with replacement (10,000 resamples) over the same per-probe records that produce the point estimates, including the published missing-score imputation; paired score differences are computed within-probe before resampling.

6 Results

6.1 Pilot Studies: Experiment 0 and Experiment 1

Experiment 0 (precision-based evaluation) failed for the reasons described in Section 5: stateless probes penalised accurate context-dependent retrieval, and a weak 7B judge hallucinated failures. The manually re-scored 31% precision confirmed that the gap was the judge, not the retrieval. This failure taught us that evaluating retrieval intermediates is the wrong level of abstraction—we must evaluate model outputs—and led directly to LSREP.

Experiment 1 (“Infant Metabolism”) was the first full LSREP run, using the pilot environment of Section 5.3: the immature system (vector + BM25 legs, MiniLM classifier, static budget, non-functional Codex), 18 conversations at isolated checkpoints, hand-written probes, and a single frozen ground truth. It was evaluated on 657 probes. After the token-accounting audit described in Section 5.3, the corrected numbers tell a sobering story. table 5 shows the six-core benchmark with corrected token counts. *We report Experiment 1 not as a result but as a pilot*—its value is diagnostic, and Experiment 2 (Section 6.2) is the paper’s substantive evaluation.

Table 5: Experiment 1 (Immature System) — Six-core benchmark, 657 probes, corrected token counts. After correcting the sliding-window under-count, ICE used more tokens than the vector baseline, scored slightly lower, hallucinated more, and had a worse TUR. The system was not yet a fair test of the architecture.

Condition	Score	Tokens	Hall. (%)	TUR	Win %
Control Baseline (Generalist)	3.05 ± 1.61	3,998	74.1	0.76	11.3
Control MoE	3.01 ± 1.61	3,998	74.4	0.75	7.6
Vector RAG (Generalist)	4.06 ± 1.31	5,847	64.7	0.69	22.9
Vector RAG (MoE)	4.05 ± 1.32	5,847	66.7	0.69	15.6
Full ICE (Generalist)	4.04 ± 1.33	8,586	69.6	0.47	22.0
Full ICE (MoE)	3.96 ± 1.33	8,586	72.3	0.46	20.6

The full ICE generalist scored 4.04 against the vector baseline’s 4.06, used 47% more tokens (8,586 vs 5,847), hallucinated more (69.6% vs 64.7%), and posted a worse TUR (0.47 vs 0.69). The MoE variant was worse still (3.96). The picture brightened slightly on the longitudinal axis—193 tracked question curves showed scores improving over time on repeated probes, demonstrating memory accumulation—but the headline numbers were a defeat. Several subsystems were broken or immature: only 3 decay cycles had run; the Codex knowledge graph was non-functional (regex-only NER with no real entity extraction); 22 gating failures meant the classifier was actively blocking retrieval on probes that needed it; the sliding-window under-count had to be corrected after the fact; and the token budget was static, not dynamic.

Lessons learned. A system that has not had time to consolidate memory is not a fair test. Honest token counting is essential—the sliding-window under-count was a measurement bug that, left uncorrected, would have made ICE look more efficient than it was. And a broken subsystem (Codex, NER) can drag down the entire system, which is itself an important finding about the fragility of multi-component architectures. Experiment 1 was not published as a standalone result; it served as a pilot that revealed every measurement flaw and system immaturity that needed to be addressed before a fair evaluation could be run.

6.2 Mature System Evaluation (Experiment 2)

Experiment 2 focused on four long-form conversations. table 6 reports the four conditions on the three-conversation “fair comparison” view (Dataset C / Technical Planning excluded; 1,057 probes). table 7 breaks the results down by conversation. table 8 shows the score distribution, and table 9 groups the scores by quality bucket.

The mean-score difference between ICE Generalist and Vector Generalist is +0.01 (4.26 vs 4.25). A paired per-probe bootstrap (10,000 resamples over the 1,057 probes; both conditions answer the same probes, so the difference is computed within-probe) places this difference at +0.00 with a 95% confidence interval of

Table 6: Experiment 2 (Mature System) — Global comparison, ICE-Dev excluded (1,057 probes across 3 conversations). On the three conversations where both systems could actually generate usable answers, ICE essentially ties the vector baseline on mean score (4.26 vs 4.25), injects 32% fewer fragments, and wins 30.6% of head-to-head comparisons (vs 21.2%).

Condition	Score	Tokens	Frag	SPF	TUR	Win %	Hall. %
Vector RAG Generalist	4.25 $\pm$ 1.09	21,025	30.0	0.14	0.20	21.2	20.5
Vector RAG MoE	4.24 $\pm$ 1.04	21,025	30.0	0.14	0.20	19.2	17.4
Full ICE Generalist	4.26 $\pm$ 1.08	22,411	20.4	0.21	0.19	30.6	19.6
Full ICE MoE	4.28 $\pm$ 0.99	22,411	20.4	0.21	0.19	29.0	19.9

Table 7: Experiment 2 — Per-conversation breakdown (generalist conditions). ICE wins more head-to-head comparisons on every conversation. On Dataset A (Creative Writing) ICE’s win rate (37.2%) is more than double the vector baseline’s (18.0%).

Dataset	ICE Score	Vec. Score	ICE Tok.	Vec. Tok.	ICE Win %	Vec. Win %
Dataset A (Creative Writing)	3.63	3.50	19,434	24,360	37.2	18.0
Dataset B (Long-Form Creative)	4.28	4.33	24,108	21,257	32.8	22.9
Dataset D (Academic Planning)	4.63	4.57	20,103	18,076	20.4	18.8

Table 8: Experiment 2 — Score distribution (1,057 probes, ICE-Dev excluded). The full distribution shows that the mean-score tie masks a structural difference: ICE-MoE has the lowest score-1 rate (1.5%) but shifts mass from score-5 to score-3/4, producing a lower-variance output.

Score	ICE Gen (%)	ICE MoE (%)	Vec. Gen (%)	Vec. MoE (%)
1 (poor)	3.0	1.5	3.8	2.4
2	5.9	4.3	4.3	3.9
3 (acceptable)	13.3	16.6	14.8	19.5
4 (good)	18.1	19.7	17.2	16.3
5 (excellent)	59.7	58.0	60.0	58.0

Table 9: Experiment 2 — Temporal score quality (1,057 probes, ICE-Dev excluded). Quality buckets: Good (4–5), OK (3), Poor (1–2). ICE-Generalist has the highest Good rate (77.8%) among the four conditions.

Condition	Good (4–5) %	OK (3) %	Poor (1–2) %
Vector RAG Generalist	77.2	14.8	8.0
Vector RAG MoE	74.3	19.5	6.2
Full ICE Generalist	77.8	13.3	8.9
Full ICE MoE	77.7	16.6	5.8

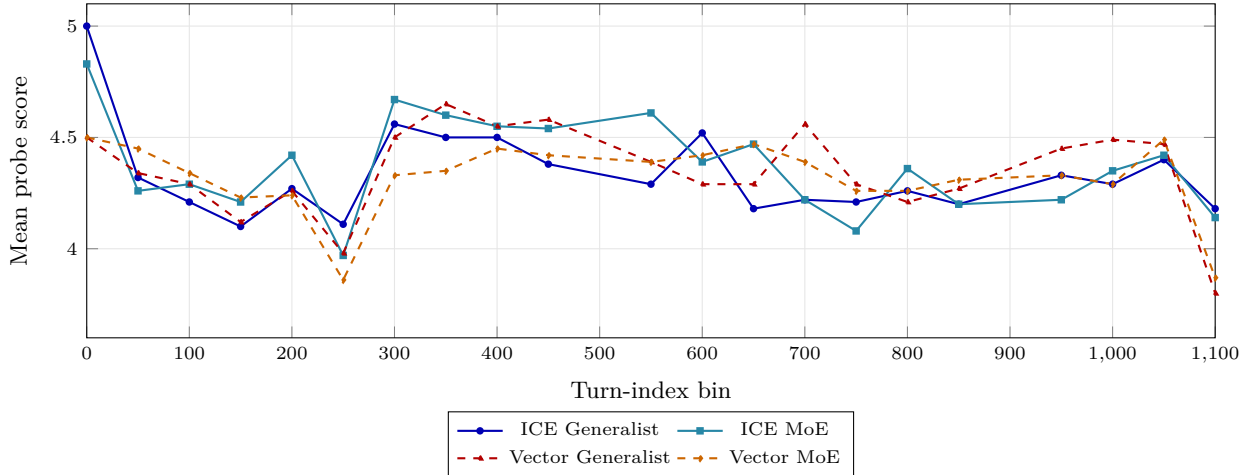


Figure 4: Longitudinal mean score by turn-index bin (1,057 probes, ICE-Dev excluded; real per-bin data). All four conditions track closely through the mid-conversation bins, but at the final turn-1,100 bin—the deep long-horizon regime of Dataset B—both vector conditions drop sharply (to 3.80 generalist, 3.87 MoE) while both ICE conditions hold (4.18 generalist, 4.14 MoE). The divergence at the extreme horizon is the practical advantage of ICE’s consolidated long-tail memory.

$[-0.07, +0.07]$: a genuine, tightly-estimated statistical tie, not an ambiguous one. Several secondary metrics tell a more interesting story. ICE injects 32.0% fewer fragments (20.4 vs 30.0) while achieving the same quality, giving an SPF of 0.21 vs 0.14 (a 50% improvement in efficiency per fragment). The head-to-head win rate is the cleanest signal, and it is statistically robust: ICE Generalist wins 30.6% of tournament appearances (95% CI [27.9, 33.4]) against the vector baseline’s 21.2% ([18.8, 23.7])—the intervals do not overlap. When the systems disagree, ICE is more likely to be right. ICE uses 6.6% more tokens (22,411 vs 21,025) because its fragments are higher-quality and longer on average, but it uses them in a more compact set. Hallucination rates are roughly equal (19.6% vs 20.5%).

Leg contributions (ICE Generalist): episodic 62.3%, codex 3.3%, unknown (fragments whose source leg could not be reconstructed) 34.5%. The codex contribution is small but targeted—and the 3.3% figure additionally *undercounts* the Codex’s share of injected content by construction: the evaluated implementation collapsed an entire graph traversal (every visited entity and its context payload) into a *single* concatenated fragment per probe, whereas the episodic legs emit each turn as its own fragment. A Codex contribution carrying a dozen entities therefore counted as one fragment against dozens of individually-counted episodic fragments, and could occupy at most one slot in token-budget enforcement regardless of relevance. The fragment-share metric is thus a lower bound on the Codex’s content share, not a measure of it. **Fragment-score correlation** reveals a qualitative difference: ICE Generalist shows $r = +0.193$ (MoE: +0.246), while Vector Generalist shows $r = -0.015$ (MoE: -0.030). For ICE, retrieving more fragments correlates with better answers; for the vector baseline, retrieving more fragments correlates with worse answers—a strong signal that ICE’s fragments carry signal while the vector baseline’s carry noise.

Per-conversation results vary by domain. On Dataset A (Creative Writing), ICE’s win rate is more than double the vector baseline’s (37.2% vs 18.0%). On Dataset B (Long-Form Creative Writing), the largest conversation at 1,119 turns, the vector baseline actually wins on mean score (4.33 vs 4.28) but ICE wins on win rate (32.8% vs 22.9%) and uses 18% more tokens because it serves more detailed, complete answers. On Dataset D (Academic Planning), ICE wins on both mean score (4.63 vs 4.57) and win rate (20.4% vs 18.8%). **Longitudinal score evolution.** fig. 4 shows how the mean probe score evolves across turn-index bins for the four conditions on the three-conversation view. All four conditions track closely through the early and middle bins—the mean-score tie is visible as four largely overlapping curves. The decisive separation appears at the deepest long-horizon bin (turn 1,100 of Dataset B): both vector conditions drop sharply to 3.80 (generalist) and 3.87 (MoE), while both ICE conditions hold at 4.18 (generalist) and 4.14 (MoE). The vector baseline is not using the long-tail knowledge that ICE has consolidated; its retrieval cannot reach across 1,000+ turns of conversation history, and the gap opens precisely where that reach matters.

MoE vs. generalist. The global MoE-vs-generalist score delta is +0.01 for ICE and -0.02 for vector; the effect is too small to claim a meaningful advantage in either direction. MoE provides a modest 3.1% hallucination reduction under vector but a -0.3% hallucination increase under ICE. MoE provides no token savings. The MoE infrastructure is therefore architecturally present but empirically neutral in this evaluation—a finding we return to in the Discussion.

Memory maturity is reported per conversation: Dataset B had 93 simulated decay days at its final checkpoint, Dataset A had 24, Dataset C had 27, Dataset D had 20. Gating failures dropped from 22 in Experiment 1 to 2 in Experiment 2—the single largest reliability improvement in the entire mature deployment.

Lessons learned. The mean-score tie masks an important qualitative difference: ICE wins more head-to-head comparisons and its fragments are positively correlated with answer quality, while the vector baseline’s fragments are negatively correlated. The 32% fragment reduction is the most practically significant finding—it means ICE can achieve the same quality with substantially less context pollution. The MoE finding is the most humbling: an architecturally elaborate component delivers no measurable benefit under realistic conditions.

6.3 Stress Test: Context Overflow on Dataset C (ICE-Dev)

Dataset C (Technical Planning) contains architecture documents that frequently exceed 8,000 tokens per turn. The vector baseline, which always retrieves the top-30 fragments with no per-query budget, injects a mean of 88,061 tokens (with failed cases reaching 100,505), overwhelming the model’s context window. Of 154 probes, 145 fail with a score of 1 (a 94.2% failure rate)—the retrieval subsystem actually found relevant information, but the generation model could not process the assembled context and produced empty or degenerate responses. ICE, with its dynamic token budget and `_enforce_token_budget` capping retrieval, maintains a mean of 19,953 tokens and produces a mean score of 4.33.

Table 10: ICE-Dev stress test (154 probes). The vector baseline collapses to a 94.2% failure rate on the technical-planning conversation because it has no token-budget cap. ICE survives the same conversation with a mean score of 4.33 and 77% fewer tokens injected. Note: the vector baseline’s 0% hallucination rate is misleading—94.2% of its responses are complete failures (score 1), so there is no answer to evaluate for hallucination.

Condition	Score	Tok.	Frag	SPF	TUR	Hall %	Win %	Overflow %
Vector RAG Generalist	1.23	88,061	30	0.04	0.01	0.0*	1.9	94.2
Vector RAG MoE	1.10	88,061	30	0.04	0.01	0.0*	4.5	97.4
Full ICE Generalist	4.33	19,953	10	0.43	0.22	41.4	43.5	3.9
Full ICE MoE	4.03	19,953	10	0.40	0.20	40.8	50.0	16.2

The score distribution on Dataset C is striking. ICE Generalist: 3.9% at score 1, 5.2% at 2, 11.7% at 3, 12.3% at 4, and 66.9% at 5—66.9% of its answers are excellent. The vector baseline: 94.2% at score 1, 0% at 2/3/4, and 5.8% at 5. ICE’s SPF is 0.43, more than $10\times$ the vector baseline’s 0.04; the same 0.43 of “quality per fragment” would mean 130 fragments for the vector baseline to match ICE’s quality, which is structurally impossible.

The 41.4% hallucination rate for ICE on Dataset C deserves a careful note. ICE is trying to answer a difficult question, and sometimes gets it wrong. The vector baseline is not trying at all (94.2% of its responses are complete failures), so it never hallucinates. A 0% hallucination rate here is not evidence of reliability; it is evidence of absence. The 41.4% hallucination rate reflects the difficulty of the task and the model’s confidence on a domain where it has more material to work with, not a failure mode unique to ICE.

Information density as a benchmark dimension. One of the most important observations from Experiment 2 is that conversational turn count alone is an insufficient measure of benchmark difficulty. Dataset B represented the greatest challenge in terms of *temporal horizon* (retrieval across 1,000+ turns); Dataset C represented a different challenge entirely: *extreme information density*. The benchmark therefore revealed two distinct stress dimensions: temporal-horizon stress and information-density stress. While Dataset

B primarily evaluated long-range memory retention, Dataset C unexpectedly became a test of context-budget management and retrieval scalability.

What the headline numbers look like with ICE-Dev included. Throughout Sections 5.4 and 6, the four-condition comparison (table 6) deliberately excludes Dataset C so that the reported 4.26-vs-4.25 tie reflects conditions under which both systems can actually produce an answer—a 94.2% failure rate would otherwise dominate any aggregate statistic without telling the reader anything about answer *quality*. For completeness, and so this choice is not mistaken for cherry-picking, table 11 reports the same four conditions computed over *all* 1,211 probes across all four conversations, ICE-Dev included.

Table 11: Experiment 2, all four conversations including ICE-Dev (1,211 probes total). Including the conversation where the vector baseline collapses does not just narrow the gap—it *reverses* it: the vector baseline’s mean score falls to 3.87 (from 4.25 excluding ICE-Dev) while ICE’s mean score rises slightly to 4.27 (from 4.26), because ICE survives the stress case the vector baseline cannot. The win-rate and fragment-efficiency gaps both widen relative to table 6.

Condition	Score	Tokens	Frag	SPF	Win %	Hall. %
Vector RAG Generalist	3.87 $\pm$ 1.47	29,550	30.0	0.13	18.7	20.2
Vector RAG MoE	3.84 $\pm$ 1.45	29,550	30.0	0.13	17.3	15.7
Full ICE Generalist	4.27 $\pm$ 1.09	22,099	19.1	0.22	32.3	22.3
Full ICE MoE	4.25 $\pm$ 1.03	22,099	19.1	0.22	31.7	22.5

On this all-data view the paired per-probe ICE-minus-vector score difference is +0.40 (95% bootstrap CI [+0.31, +0.49]), decisively excluding zero; on the ICE-Dev stress test alone it is +3.10 ([+2.85, +3.33]). This is, if anything, a stronger result than the headline tie suggests, and we want to be explicit about why we did not lead with it. Excluding ICE-Dev from the primary comparison is the more conservative choice, not the more flattering one: it isolates the "does ICE cost anything when both systems can answer" question from the "does ICE survive when the baseline cannot" question, so that a reader can evaluate each claim on its own evidence rather than one number smuggling in the other. Reporting only the all-data table would invite a fair objection that a single catastrophic-failure conversation is doing all the work of the headline result; reporting only the no-ICE-Dev table would omit a real and important finding. table 10 exists precisely so the ICE-Dev result is visible and auditable on its own, and table 11 exists so a reader can see exactly how much the exclusion in table 6 was doing. We consider both tables necessary and neither sufficient alone.

Lessons learned. The token budget is ICE’s most important safety mechanism. Without it, any retrieval system—no matter how sophisticated—will collapse on conversations with large per-turn documents. The fact that ICE achieves its highest absolute score on the exact conversation where the baseline collapses (4.33) demonstrates that retrieval quality is not sacrificed by the budget; it is improved by it. This stress test was unplanned—it emerged during the evaluation run when the first few probes returned errors—but it produced the single strongest result in the entire evaluation: a 94% failure rate eliminated by a per-query token budget.

6.4 Feature Ablation: Cumulative Buildup (Experiment 3)

To understand *why* ICE works, we ran a feature-buildup ablation on the Long-Form Creative Writing dataset (Dataset B, 67 probes, fully mature memory state, single-pass, judged with Qwen3-14B-AWQ because the 14B model was the only one available on the dedicated inference server without interrupting the main evaluation). The system was constructed incrementally from a bare-vector baseline, with each step adding exactly one feature and preserving all previously enabled components.

Several findings stand out. **RRF is the single most impactful component** (+0.84). Adding RRF after BM25 rescues the system from the BM25 damage and pushes it above the bare-vector baseline. The mechanism is clear: BM25 adds noisy lexical matches that need the fusion quality signal of RRF to be useful. **BM25 alone is destructive** (−0.75). Adding BM25 without RRF floods the context with low-quality lexical matches that actively hurt answer quality. This is a cautionary finding: naive multi-leg retrieval without fusion is worse than single-leg retrieval.

Table 12: Cumulative feature addition on Dataset B (67 probes). RRF (+0.84) is the single most impactful step; BM25 alone (−0.75) is the single most destructive. Several intuitively helpful features—HyDE, cluster restriction, session diversification, MERA—are neutral or negative. Full ICE finishes +0.11 above bare vector but 2× more efficient per fragment. The vector baseline row is included for reference (it is not a step in the buildup).

Step	Score	Step Δ	Cum. Δ	SPF	Tokens	Frag	Rec. Δ
bare_vector	3.27	—	0.00	0.23	14,873	14.0	+0.42
+BM25	2.52	−0.75	−0.75	0.17	14,868	15.2	+0.29
+RRF	3.36	+0.84	+0.09	0.29	14,870	11.7	+0.54
+HyDE	3.39	+0.03	+0.12	0.29	14,870	11.6	+0.52
+cluster_restrict	3.40	+0.01	+0.13	0.30	14,911	11.3	+0.66
+session_diversify	3.40	+0.00	+0.13	0.30	14,911	11.2	+0.54
+codex	3.43	+0.03	+0.16	0.30	14,912	11.3	−0.14
+MERA	3.22	−0.21	−0.05	0.28	14,912	11.4	+0.17
+procedural	3.39	+0.17	+0.12	0.30	14,912	11.4	−0.10
+batch_summary	3.41	+0.02	+0.14	0.30	14,912	11.3	+0.45
+dynamic_budget	3.30	−0.11	+0.03	0.16	24,489	21.2	+0.08
+sliding_window	3.32	+0.02	+0.05	0.16	27,763	21.3	+0.30
+keyword_boost	3.44	+0.12	+0.17	0.23	27,769	15.1	+0.70
full_ice	3.38	−0.06	+0.11	0.22	27,768	15.0	+0.37
vector_baseline (ref.)	3.42	—	+0.15	0.11	16,373	30.0	−0.03

HyDE, cluster restriction, and session diversification are neutral ($\Delta \leq 0.03$). These features do not hurt, but they do not help either—at least not on a creative-writing dataset. They may help in other domains. **MERA is actively harmful** (−0.21). The Meta-Enumeration Retrieval Agent injects category-level entity lists that add noise without improving answer quality. On a creative-writing dataset, the user rarely asks “list all the characters”—they ask specific questions about specific characters. **Codex has a small positive effect** (+0.03) but a *negative* recency delta (−0.14), meaning Codex helps early-turn questions (by providing structured entity context from the beginning) but does not improve over time. **Dynamic budget is negative** (−0.11) on this dataset: the budget formula allocates more tokens to longer conversations, which *hurts* on Dataset B because it injects too many fragments (21.2 vs 11.3 at the pre-budget step). This is counter-intuitive and suggests that the budget formula needs domain-specific tuning.

Table 13: Recency effect by origin split (fact age). Probes grouped by the turn index at which they were created. Recency $\Delta = \text{mean}(\text{late}) - \text{mean}(\text{early})$. ICE-with-RRF (and its later descendants) consistently shows positive recency, meaning answers improve as the conversation grows; the vector baseline shows a slightly negative recency, meaning it does not.

Condition	Early (0–400)	Mid (400–800)	Late (800–1200)	Recency Δ
bare_vector	3.00	2.83	3.42	+0.42
+BM25	2.20	3.05	2.49	+0.29
+RRF	3.00	2.83	3.54	+0.54
+keyword_boost	2.90	3.28	3.60	+0.70
full_ice	3.10	3.25	3.47	+0.37
vector_baseline	3.50	3.15	3.47	−0.03

The full-ICE configuration finishes 3.38, slightly above the bare-vector baseline of 3.27 but slightly below the vector-baseline reference of 3.42. The single-domain nature of this experiment (Dataset B only, 67 probes,

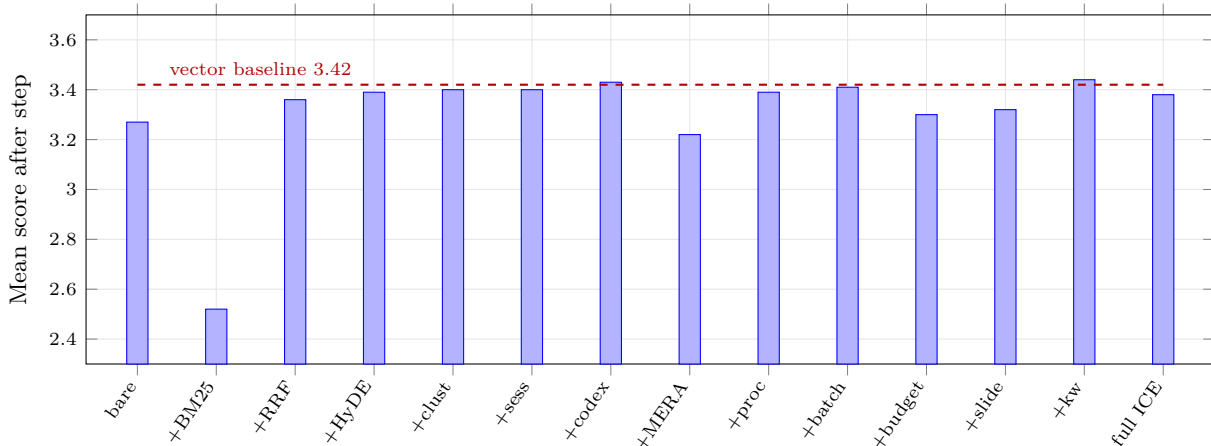


Figure 5: Feature ablation buildup (Dataset B, 67 probes, Experiment 3). Each bar is the cumulative mean score after adding one feature to all previous ones. The single largest positive swing is +RRF (+0.84, from 2.52 to 3.36); the single largest negative is +BM25 alone (−0.75) and +MERA (−0.21). The dashed line marks the single-leg vector baseline (3.42), which is not a step in the buildup. RRF is the dominant single feature.

smaller 14B model) limits what the absolute numbers can tell us, but the relative deltas are valid: RRF matters, BM25 is dangerous without fusion, MERA hurts, and several intuitively-helpful features are neutral.

Lessons learned. Not all features are created equal. RRF is the linchpin: without it, multi-leg retrieval is actively harmful. Several “obvious” improvements (HyDE, MERA, dynamic budget) are neutral or negative, which is a humbling reminder that intuition is a poor guide in system design. The right way to build a retrieval system is not to throw every feature at the wall, but to carefully measure each one’s contribution and keep only what helps.

6.5 Comparative Summary

Across the four experiments, the story is not “ICE is always better”—it is “ICE is more resilient and more efficient, and its advantage grows with conversation difficulty.” Experiment 1 showed ICE could be worse than the baseline; Experiment 2 showed it can match quality with 32% fewer fragments; the ICE-Dev stress test showed it can survive where the baseline collapses; the ablation showed *why* it works (RRF) and where it does not (MERA, untuned dynamic budget, NER-broken Codex). The narrative is one of progressive understanding rather than single-result triumph.

7 Discussion

7.1 What We Learned About Conversational Memory

Memory is not just retrieval—it is *curation*. ICE’s advantage comes not from retrieving more (it retrieves less) but from retrieving better. The token budget, the RRF fusion, the decay mechanics, the cluster scoping, the session diversification, and the keyword boost all serve to filter noise. A memory system that remembers everything is a memory system that remembers nothing useful. The fragment–score correlation is the cleanest empirical statement of this principle: for ICE, more fragments correlate positively with quality ($r = +0.19$); for the vector baseline, more fragments correlate negatively with quality ($r = -0.02$). Same metric, opposite sign.

7.2 Why Do Some Features Help in One Domain but Not Another?

The ablation revealed that several “obviously helpful” features are neutral or negative on creative-writing data. MERA is designed for categorical queries (“list all characters”) that are common in technical domains but rare in creative writing. Cluster restriction assumes topical coherence that breaks down in meandering

creative conversations. Dynamic budget assumes that longer conversations need more retrieval, which is true for technical planning but false for creative writing (where recent turns are often the most important). The lesson: retrieval features are *domain-specific*, and a good system must adapt its feature set to the conversation type—which is exactly what ICE’s classification engine attempts to do through dynamic leg weighting. The classification-driven routing is, in our view, the most under-explored design lever in the system.

7.3 How Do the Findings Generalise?

The token-budget finding (ICE-Dev stress test) should generalise to any system that injects retrieved context into a fixed-context-window model. The RRF finding (ablation) should generalise to any multi-leg retrieval system. The domain-specificity finding (MERA, HyDE neutral or negative in creative writing) is a caution against one-size-fits-all feature sets. The fragment–score correlation finding is, we suspect, universal: a system whose fragments correlate negatively with answer quality is, in a deep sense, a system that does not know what it is doing.

8 Limitations

We are deliberately specific about what this study does and does not show. Several of these limitations directly motivate the future work of Section 9.

8.1 Single-User Evaluation

All benchmark conversations were authored by a single user. While the selected conversations span creative writing, technical planning, academic planning, and long-horizon worldbuilding, they still reflect the habits, vocabulary, and interaction style of one individual. The results therefore demonstrate effectiveness for a diverse set of conversation *types* rather than for a diverse *population of users*. Repetition of this evaluation across multiple users is a prerequisite for generalising the findings beyond the current study.

8.2 Synthetic Longitudinal Reconstruction and Missing Reinforcement Loops

The evaluation reconstructs conversation history by replaying existing conversations and periodically probing the resulting memory state. This allows controlled experimentation over hundreds or thousands of turns, but cannot fully replicate the feedback loops that occur during live usage. In real deployments, retrieval events continuously influence future memory states through reinforcement, decay adjustment, bookmarking, procedural extraction, and user behaviour; during evaluation, many of these interactions are absent because future turns already exist and cannot be influenced by retrieved outputs.

More specifically, ICE includes retrieval-strengthening mechanisms that increase the `access_count` and partially restore the `decay_score` of frequently accessed memories, while allowing unused memories to decay. During evaluation, retrieval reinforcement is driven almost entirely by benchmark probes rather than by natural user interaction. In a real conversation, a user might ask about a character’s motivations every few sessions, reinforcing that memory repeatedly; in the evaluation, the memory receives reinforcement only when a probe explicitly targets it. Important concepts that would normally be revisited repeatedly during a live conversation therefore receive less reinforcement than they would in production usage, creating a more difficult retrieval environment than would be encountered during normal operation. Consequently, the benchmark likely *underestimates* the benefits of reinforcement-driven memory maturation. The positive longitudinal curves observed in Experiment 2 are promising, but are likely a lower bound on real-world performance improvement.

8.3 Routing Bias and Classifier Conservatism

Between Experiment 1 and Experiment 2, the classifier was retrained with a substantially improved embedding backbone (Qwen3-Embedding-0.6B replacing MiniLM), which reduced gating failures from 22 to 2. However, a second change was also introduced: a conservative routing override applied *after* classification. Queries

classified as `Zero_Shot` were automatically re-routed to `Long_Term_Memory` when confidence fell below a threshold, or when the conversation exceeded a certain turn count (the “long-conversation LTM bias”).

One concrete failure mode motivated part of this override: a **public-entity trap**, in which the classifier learned to associate well-known proper nouns with `Zero_Shot` (the model already “knows” who or what they are), even when the query was clearly scoped to the user’s own conversation about that entity. A question referencing a famous, widely-known character name from Dataset A’s fan-fiction work, for instance, was liable to be classified as requiring no retrieval at all—because the classifier had learned that the entity itself is public knowledge, not that the *conversation* about it is the user’s own long-running creative project. This was patched with a hard override forcing `Long_Term_Memory` for any conversation-scoped query, which is effective but crude: it does not distinguish between a public entity discussed in a genuinely stateless way and a public entity that is the subject of extensive private, user-specific elaboration. A more principled fix would require the classifier to condition on conversation history rather than the query text alone—which the context-aware classification path (Section 4) partially addresses, but not completely.

This override is a deliberate safety net: false negatives (failing to retrieve when memory is needed) are fatal, while false positives (retrieving when not strictly needed) are merely expensive. The empirical evidence from Experiment 1 supported this design—gating failures caused complete answer failures, whereas unnecessary retrieval only added modest context overhead. However, the override introduces a fundamental limitation: the system is no longer purely intent-aware in the way the architecture claims. ICE frequently forces retrieval even when the classifier predicts zero-shot, because the heuristic overrides the classifier’s judgment. The system is therefore “safe” rather than “smart”—it errs on the side of retrieval to avoid catastrophic forgetting, at the cost of occasionally injecting irrelevant context. The true strength of the classifier is thus masked by the override; we cannot claim that ICE’s intent-gating is purely driven by the MLP, because the rule-based override is actively compensating for classifier uncertainty. A more principled approach would treat the classifier’s confidence as a prior and combine it with a conversation-length prior via Bayesian inference, rather than applying a hard threshold.

8.4 Codex Extraction Reliability

The Codex subsystem is the most architecturally ambitious component of ICE and, in the current implementation, also the most fragile. Its marginal contribution in Experiment 2 (only 3.3% of retrieved fragments, +0.03 in the ablation) does not reflect the subsystem’s theoretical ceiling, but rather the cumulative effect of several simultaneous engineering handicaps.

The chunking paradox. Current extraction uses 6,000-token chunks with a 200-token overlap, chosen to avoid splitting sentences and to preserve broad narrative context. However, a 4B-parameter model does not possess the effective reasoning capacity to process 6,000 tokens simultaneously; beyond 1,000–1,500 tokens, attention becomes diluted, losing track of entities mentioned in the middle of the chunk while disproportionately weighting the beginning and end. This causes two failure modes: (1) *entity dropping*, where entities introduced mid-passage are never extracted; and (2) *relationship hallucination*, where the model confuses the subject and object of distant clauses, outputting syntactically valid but semantically meaningless triplets (e.g., `fastapi uses fastapi`). The optimal chunk size for reasoning with a 4B model is closer to 512–800 tokens, which was not used in the current evaluation.

Weak confidence and strength signals. Edges carry a `confidence` flag in {pending, active} and a numeric `strength` that decays over time, but both are under-utilised in retrieval. The Codex traversal follows any edge with `valid_until` IS NULL, regardless of confidence or strength, and the only effect of `confidence == 'active'` is a coarse 1.5× score boost at the fragment level. There is no reinforcement loop that strengthens edges on retrieval, so frequently referenced facts do not rise in the ranking, and traversal treats weak or decayed edges as equally important as strong ones. Edge strength and confidence are therefore almost-inert signals that contribute little to retrieval quality.

Semantic versus lexical matching. Codex retrieval is exact-match (canonical name or aliases) plus vector fuzzy matching over the `context_payload` embedding, not the entity name itself. If a user establishes an entity “The Obsidian Citadel” and later asks “where is the main fortress located?”, the entity resolution step has no lexical or embedded anchor for “fortress” to match against—even though a human reader immediately

understands the reference. This is a fundamental limitation of entity-centric retrieval in free-form conversation: it assumes users consistently refer to entities by canonical names or close aliases, which is empirically false in natural dialogue.

Implications. The marginal contribution of Codex should be read as a floor, not a ceiling. That it produced any positive contribution despite oversized chunks, slow corroboration, and an underpowered extractor suggests that a properly implemented Codex could become one of the strongest components in the architecture. The negative result for MERA (-0.21) is similarly contextual: MERA activates only when entity extraction finds no entities, so it is a downstream indicator of extraction gaps rather than an independent subsystem evaluation.

8.5 Mixture-of-Experts Routing: Hardcoded, Untrained, and Latency-Bounded

The MoE-vs-generalist comparison shows negligible differences (global $\Delta = +0.01$ for ICE, -0.02 for vector). The implemented router has three deficiencies that together explain this. First, routing is based on hardcoded topic/intent mappings rather than learned or empirical model-performance data; there is no signal that a model tagged `Software_&_Tech` actually outperforms a generalist on software questions. Second, the router ignores classifier confidence—a `max_confidence` of 0.55 is treated identically to 0.95. Third, it ignores context-reliance labels, routing self-contained `Zero_Shot` queries identically to context-heavy `Long_Term_Memory` queries. An operational cost compounds these: when the router selects an unloaded model, Ollama unloads the current model and loads the new one (5–15 s), and ICE has no control over this process. The MoE infrastructure is therefore both conceptually underdeveloped and operationally costly.

8.6 KV-Cache Optimisation: Designed but Largely Ineffective in Practice

The Prompt Assembler uses stable-prefix ordering to maximise KV-cache reuse. In practice, cache hits are rare: the recent-turn window changes on every request (invalidating the prefix from that point on), the retrieved-context block changes on most queries, MoE model swaps wipe the cache entirely, and Ollama’s cache is ephemeral and session-bound. The only reliably cacheable segments—the system message and persistent slots—account for roughly 10–15% of the prompt. Stable-prefix ordering is the correct *design*, but the practical benefit is limited by factors outside ICE’s control.

8.7 Judge Model Limitations and the Experiment 3 Audit Gap

Evaluation relied primarily on an automated judge (`mattbucci/gemma-4-12B-AWQ`). During analysis, multiple cases were identified where correct, true information was incorrectly flagged as hallucinated because it was absent from the *condensed* ground-truth summary rather than from the conversation itself—a systematic over-harshness that affected all four evaluated conditions equally, not a bias against any one system. To correct this, every hallucination flag across all 1,211 Experiment 2 probes was individually audited against the source conversation and corrected where the flagged detail was verifiably true; flags on genuine fabrications were left in place, and no credit was given for speculative or unverifiable additions. This is label correction, not score substitution—the judge’s own criteria were applied more carefully, not replaced with human opinion. Experiment 3 (the ablation) was *not* audited this way: because it evaluated fifteen closely related variants under identical conditions, manual adjustments would have risked introducing subjective bias across comparisons that are meant to isolate a single architectural change at a time. Its absolute hallucination values (e.g., the 41.4% ICE-Dev rate) should therefore be interpreted cautiously as potentially inflated by the same automated-judge false positives described above; the *relative* differences between conditions remain valid, but the absolute numbers should not be compared directly to Experiment 2’s audited values.

8.8 Hallucination Rates and the ICE-Dev Paradox

ICE exhibits higher hallucination on Dataset C (41.4% generalist) than globally (20.2%), despite achieving its highest absolute score there (4.33). The apparent contradiction is resolved by the denominator: the vector baseline failed on 94.2% of ICE-Dev probes (no answer), yielding a 0% hallucination rate *by definition*. ICE,

because it survives these probes, generates answers and therefore has opportunities to hallucinate. The rate reflects the difficulty of the task, not a failure mode unique to ICE.

8.9 Further Scope Limitations

Probe generation bias. Most probes were generated through a constrained LLM-assisted pipeline; generated questions may differ from the more context-dependent, anaphoric, pragmatically ambiguous questions real users ask. This is partially mitigated by 72 manually authored probes carried over from earlier experiments, but the benchmark remains weighted toward automatically generated probes. **Domain coverage.** The four domains omit collaborative software development, multi-user conversations, customer support, multilingual dialogue, and workplace communication, where multi-party dynamics or formal structures may change performance. **Cross-conversation retrieval.** ICE contains infrastructure for cross-conversation search, but every probe targets a single conversation; within-conversation memory is demonstrated, cross-conversation aggregation is not yet validated. **Decay horizon.** The longest simulated horizon is 93 days; year-scale stability, saturation, and the eventual convergence of decay dynamics remain open. **Hardware-constrained ablation.** All experiments ran on a single 24GB RTX 5090. The Experiment 3 ablation used Qwen3-14B-AWQ rather than the 26B generalist to fit 15 conditions $\times$ 67 probes in a reasonable time; relative deltas hold, but absolute scores are systematically lower than Experiment 2’s, and the system has not been evaluated on enterprise-grade (80GB+) hardware. **No comparison with commercial systems.** ChatGPT’s memory and Claude’s project features cannot be run locally with controlled retrieval; the vector-RAG baseline is the strongest fair comparison constructible on identical hardware.

9 Future Work

The limitations above define a concrete agenda. We group planned work into extraction and grounding, autonomous maintenance, cross-conversation memory, retrieval under stress, explainability, learned routing, and conversation import.

9.1 Self-Correcting Codex Extraction

The current Codex is a passive triplet collector. We plan a four-stage self-correcting pipeline: (1) *deterministic NER grounding* on CPU, reusing the NER model in post-flight to produce a confirmed entity list and bypass LLM hallucination for entity identification; (2) *relationship mapping* on GPU, where the LLM receives the text *and* the confirmed entity list and only maps relationships between those entities, drastically reducing hallucinations like `fastapi` uses `fastapi`; (3) *deterministic validation*, where every proposed triplet is checked against the existing graph and contradictions trigger a reconciliation loop rather than a blind write, turning the extractor into an active state reconciler; and (4) *confidence-calibrated storage*, where edges carry a continuous extraction confidence rather than a binary `pending/active` flag. For code-heavy conversations, a deterministic static-analysis layer (AST parsing, import-graph construction, call tracking) will produce edges (`imports`, `calls`, `inherits`, `defined_in`) requiring no LLM extraction at all.

9.2 Agentic Background Maintenance

Current background workers run fixed pipelines and cannot react to cross-cutting inconsistencies. We plan to replace individual extraction workers with a *Memory Maintenance Agent*—a lightweight LLM given a toolset (`update_entity_relation`, `merge_conflicting_entities`, `reconcile_graph_state`, `flag_for_review`, `run_cluster_consolidation`) and tasked with maintaining memory consistency during idle GPU time. The agent would decide autonomously whether to extract, check for contradictions, merge entities, or escalate to the review queue, shifting the background worker from a deterministic script executor to an autonomous decision-maker while preserving the review queue as a safety net. The currently skeletal Sentinel system would be fully integrated, with the agent subscribing to Sentinel events and proactively resolving flagged issues.

9.3 Cross-Conversation Project-State Memory and Coding Mode

We plan to extend ICE beyond single-conversation memory via a *Project State Engine*: dedicated tables and workers tracking architecture clusters, decisions, tasks, and git history alongside the conversational stores. Each project maintains its own memory scope; retrieval operates across a project’s entire history under a per-project token budget. This enables coding-mode retrieval (surfacing relevant code files and architecture decisions across all project conversations), decision tracking (critical decisions become Codex edges with explicit temporal provenance), git integration (each commit treated as a timestamped fact scanned by a state-reconciler agent), and task management (the `pending_items` slot elevated to a full task table with status tracking).

9.4 Retrieval Under Extreme Context Density

The ICE-Dev stress test motivates retrieval methods for extremely dense contexts: *chunk-aware retrieval* that operates at the 512-token chunk level rather than the turn level, preventing a single 8,000-token document from dominating the budget; *hierarchical summarisation* producing a three-layer abstraction (100-token abstract, 500-token summary, full document) so retrieval surfaces the cheapest sufficient layer; *document decomposition* into section-level chunks with independent embeddings; and *context-adaptive truncation* that prioritises relevant chunks over whole documents, replacing the current heuristic truncation with a learned policy.

9.5 Unified, User-Controlled Context Budgeting

The dynamic token budget is currently opaque. We plan to expose it through the frontend: a retrieval-aggressiveness slider (capped / moderate / aggressive), a force-wide-net “deep search” toggle, per-project budget overrides, and a real-time telemetry panel showing how budget was allocated across legs for the current query. This turns the budget from a hidden heuristic into a user-controlled resource and creates a natural sandbox for learning optimal policies from user feedback.

9.6 User-Defined Scoping and Cross-Conversation Linking

The scoping system (None/Auto/Project/Manual) will be extended with temporary retrieval scopes (marking a turn, cluster, or document as relevant for the next N queries), manual and suggested conversation linking (based on entity overlap and topic continuity), cross-project entity relevance, and selective memory exclusion—all user-controlled, preserving user authority over memory while letting the system assist with relevance inference.

9.7 Learned Mixture-of-Experts Routing

The hardcoded overlap scorer will be replaced with a learned policy incorporating classifier confidence, context-reliance labels, and empirical model-performance data. Candidate approaches include confidence-weighted routing, context-reliance-aware routing (*Zero_Shot* to smaller faster models, *Long_Term_Memory* to larger ones), a lightweight bandit tracking per-topic model performance, and model-loading integration that pre-loads the selected model to eliminate the 5–15 s switch latency. The LSREP protocol can itself be repurposed to learn these policies offline by replaying historical probes under different routing policies and measuring answer quality.

9.8 Long-Horizon Studies, Explainability, and KV-Cache Persistence

Three further directions follow directly from the limitations. *Year-scale deployments* will study memory saturation, retrieval drift, long-term decay dynamics, and compaction frequency beyond the current 93-day horizon. An *explainability (forensics) layer* will expose retrieval attribution (which leg produced each fragment and why), memory provenance (every fragment traceable to its origin turn via `source_batch_id`), a visual Codex graph with editable edges, and queryable audit trails—essential for trust in a system that evolves autonomously. *KV-cache persistence* will investigate storing precomputed KV tensors for stable prefix

segments on disk, cache-aware retrieval policies that prefer already-cached fragments, and incremental cache updates, as inference engines expose more cache-control APIs.

9.9 User-Guided Continual Learning

An opt-in feedback system will let users rate responses: a thumbs-up adds the query and classification to the curated dataset; a thumbs-down triggers re-classification by a stronger model, with the corrected label added to the dataset. The weekly fine-tune worker consumes this dataset and retraining the classifier head, and a promotion script closes the currently manual loop by swapping the checkpoint and restarting the proxy. A manual-correction side panel lets experienced users edit predicted labels directly. The system is disabled by default, with explicit opt-in and warnings about the risk of degrading the classifier with inconsistent feedback.

9.10 Conversation Import as a First-Class Feature

LSREP, developed as a research benchmark, is in effect a complete conversation-ingestion pipeline: the same chronological replay, post-flight evaluation, Codex extraction, procedural extraction, clustering, and reflection used for evaluation can ingest exported chat logs. This directly attacks the vendor lock-in of memory—users with months or years of history on cloud platforms could export it, drop it into an ingestion endpoint, and obtain a fully mature ICE deployment that treats the imported history as if it had always been present. A key design lever is decay scaling for imported history: rather than applying months of decay on arrival, ingested turns receive a configurable stabilisation window (a `decay_immune` flag), an optional fast-forward mode that simulates the original timeline, and user-selectable ingestion policies (preserve all / simulate natural decay / start fresh). Because ICE reconstructs episodic memory, a knowledge graph, procedural memory, and clusters as distinct stores, import means reconstructing the entire cognitive state the system would have had if present during the original conversation—not merely building a searchable archive, as a naive vector-only import would. The imported history is not just retrievable text; it is *lived through* by the background workers, producing the same structured memory state that would have emerged from live interaction.

10 Conclusion

We have presented the Infinite Context Engine, a self-hosted memory middleware with a multi-store architecture, intent-driven retrieval routing, a temporally-versioned knowledge graph, and a per-query dynamic token budget. We have presented LSREP, a longitudinal evaluation protocol that emerged from the failure of a precision-based metric and that measures not just retrieval quality at a single point in time, but memory accumulation, retention, and belief revision across months of conversational history. We have presented four experiments that, taken together, show ICE matches vector-RAG quality on long-form conversations while injecting 32% fewer fragments, wins a substantially larger share of head-to-head comparisons, and survives a context-overflow stress test where the vector baseline collapses to a 94.2% failure rate. We have presented a feature-buildup ablation that identifies RRF as the single most impactful component, that reveals several intuitively-helpful features to be neutral or negative, and that gives a quantitative roadmap for future development.

As context windows grow, the temptation is to simply stuff more context in. Our results suggest the opposite: less-but-better context outperforms more-but-noisier context in long-horizon conversational AI. The future of conversational memory is not bigger windows—it is smarter curation. ICE’s architecture, with its classification-driven routing, RRF-based fusion, and per-query token budget, points toward a design philosophy where the memory system acts as a *curator* rather than a *firehose*. We hope this work contributes to that philosophy and that the released implementation, evaluation protocol, and ablation harness enable others to build on it.

Broader Impact and Reproducibility

The substantive limitations of this work are enumerated in full in Section 8: single-user evaluation, synthetic longitudinal reconstruction, classifier-override conservatism, Codex extraction fragility, the neutrality of the

MoE router, ineffective KV-caching in practice, the Experiment 3 audit gap, and constrained domain and hardware coverage. We add two further observations on reproducibility and ethics.

Reproducibility. The full source code, the evaluation harness, and the experimental configuration are released as part of this work. All reported results reflect the system as it existed at the evaluation-time snapshot, preserved in the repository as the git tag `v2-paper-eval` (the tag preserves the evaluated *code*; companion artifacts written after the snapshot—this paper and the archived technical report—live on the repository’s main branch); the implementation has continued to evolve since. All numbers reported in Section 6 are derived from publicly shareable JSON reports; the raw judge-model outputs and the LSREP probe sets are not released because they contain personal conversational content. A distinction is worth stating plainly: the *protocol* is fully reproducible, but the *data* is not manufacturable. The four corpora are real conversations accumulated through genuine use over months to years—Dataset B alone represents roughly a year of actual interaction—and their evaluation value derives precisely from that authenticity: the entity evolution, contradictions, superseded decisions, and long-range dependencies are organic, not planted. An equivalent corpus cannot be synthesised on demand, and this is a general property of longitudinal memory evaluation rather than a limitation specific to this study. Reproducing the evaluation therefore means bringing one’s own long-form conversational history (most platforms support chat-log export, which the replay harness ingests); the protocol, harness, judge configuration, and metrics transfer unchanged to any such corpus.

Ethical considerations. All four evaluation datasets were created by the first author, who is the sole data subject. No third-party conversational data was used. The system stores the user’s own data on the user’s own hardware; no data leaves the local machine. The architecture includes a memory-slot system in which high-stakes steering slots (project context, user preferences, guidance) require explicit user approval before Reflection-proposed updates are applied, while low-stakes bookkeeping (pending items) is autonomous. This human-in-the-loop boundary is a deliberate design choice rather than a performance limitation.

References

- Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. 2023. MemGPT: Towards LLMs as Operating Systems. *arXiv preprint*. Available at <https://www.semanticscholar.org/paper/MemGPT%3A-Towards-LLMs-as-Operating-Systems-Packer-Fang/908dad62c0e43d80e3e3cb3c0402f7c71c70499c>.
- Prateek Chhikara, Abhinav Khant, et al. 2025. Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory. *arXiv preprint*, arXiv:2504.19413. Available at <https://arxiv.org/abs/2504.19413>.
- Wanjun Zhong, Lianghong Guo, et al. 2024. MemoryBank: Enhancing Large Language Models with Long-Term Memory. *Proceedings of the AAAI Conference on Artificial Intelligence*. Available at <https://ojs.aaai.org/index.php/AAAI/article/view/29946>.
- Chufan Shi, Cheng Yang, et al. 2024. Unchosen Experts Can Contribute Too: Unleashing MoE Models’ Power by Self-Contrast. *arXiv preprint*, arXiv:2405.14507. Available at <https://arxiv.org/abs/2405.14507>.
- Weilin Cai, Juyong Jiang, Le Qin, Junwei Cui, Sunghun Kim, and Jiayi Huang. 2024. Shortcut-connected Expert Parallelism for Accelerating Mixture-of-Experts. *arXiv preprint*, arXiv:2404.05019. Available at <https://arxiv.org/html/2404.05019v3>.
- Darren Edge et al. 2024. From Local to Global: A Graph RAG Approach to Query-Focused Summarization. *Microsoft Research*. Available at <https://www.microsoft.com/en-us/research/publication/from-local-to-global-a-graph-rag-approach-to-query-focused-summarization/>.
- Yu Wang, Nedim Lipka, Ryan A. Rossi, Alexa Siu, Rui Zhang, and Tyler Derr. 2023. Knowledge Graph Prompting for Multi-Document Question Answering. *arXiv preprint*, arXiv:2308.11730. Available at <https://arxiv.org/abs/2308.11730>.
- Jiashuo Sun et al. 2024. Think-on-Graph: Deep and Responsible Reasoning of Large Language Model on Knowledge Graph. *ICLR Proceedings*. Available at https://proceedings.iclr.cc/paper_files/paper/2024/file/10a6bdcabbd5a3d36b760daa295f63c1-Paper-Conference.pdf.

- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*. Available at https://proceedings.neurips.cc/paper_files/paper/2020.
- Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Ming-Wei Chang. 2020. REALM: Retrieval-Augmented Language Model Pre-Training. *Proceedings of the 37th International Conference on Machine Learning (ICML 2020)*, PMLR 119:3929–3938. Available at <https://proceedings.mlr.press/v119/guu20a.html>.
- Gautier Izacard and Edouard Grave. 2021. Leveraging Passage Retrieval with Generative Models for Open Domain Question Answering. *ACL Anthology*, EACL. Available at <https://aclanthology.org/2021.eacl-main.74/>.
- Weijia Shi, Sewon Min, Michihiro Yasunaga, Minjoon Seo, Rich James, Mike Lewis, Luke Zettlemoyer, and Wen-tau Yih. 2024. REPLUG: Retrieval-Augmented Black-Box Language Models. *Proceedings of NAACL 2024*, pages 8371–8384. Available at <https://aclanthology.org/2024.naacl-long.463/>.
- Shi-Qi Yan, Jia-Chen Gu, Yun Zhu, and Zhen-Hua Ling. 2024. Corrective Retrieval Augmented Generation. *arXiv preprint*, arXiv:2401.15884. Available at <https://arxiv.org/abs/2401.15884>.
- Stephen Robertson and Hugo Zaragoza. 2009. The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval*. Available at https://www.staff.city.ac.uk/~sbrp622/papers/foundations_bm25_review.pdf.
- Gordon V. Cormack, Charles L. A. Clarke, and Stefan Büttcher. 2009. Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods. *SIGIR '09: Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 758–759. Available at <https://doi.org/10.1145/1571941.1572114>.
- Luyu Gao, Xueguang Ma, Jimmy Lin, and Jamie Callan. 2023. Precise Zero-Shot Dense Retrieval without Relevance Labels. *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL 2023)*, pages 1762–1777. Available at <https://aclanthology.org/2023.acl-long.99/>.

A Implementation Reference

This appendix collects the operational detail that supports the architecture of Section 4 but is not required to follow the paper’s main argument. It is reproduced here for readers reimplementing the system.

A.1 Background Worker Cluster

All GPU-touching workers gate on `is_gpu_busy()` (a 20%-utilisation threshold polled via `nvidia-smi`) and, in shared mode, additionally on `is_user_active()` (the Redis key `ice:last_chat_completed` within a 10-second idle window). Retry countdowns are fixed: 15 s post-flight, 30 s extractors, 60 s decay/sentinel/compaction/reflection, 120 s batch-summariser/cluster-merge. All workers are idempotent through an `idempotency_key` UNIQUE column, turning Celery’s at-least-once delivery into effectively-once side effects.

A.2 Operational Infrastructure

The system is a single FastAPI service (`/v1/chat/completions` for the OpenAI-compatible endpoint plus routers for memory slots, user control, and model registry), backed by PostgreSQL with the pgvector extension (one `Vector(384)` column per vector table because the embedder is shared), and a Celery worker fleet over Redis. The configuration system is a Pydantic `Settings` reading from `.env`. The active inference path loads `ice_classifier_v3_qwen_ft3.pt`; classifier fine-tuning writes a timestamped file but does not auto-promote it, so production promotion requires a manual file swap. SSE event types (`classified`, `retrieval`, `context_ready`, `generating`, `degraded`) drive the observability panel and downstream monitoring.

A complete technical reference, including the full training pipeline for the classifier and NER models, the controlled relation vocabulary in its entirety, the complete worker implementation, GPU resource management,

Table 14: Background workers. The cluster is self-maintaining—no operator intervention is required for the memory lifecycle.

Worker	Trigger	Function
Post-Flight Evaluator	every turn	lossless detection, summary, dispatch extractors
Codex Extractor	lossless turns	triplet extraction, handle_triplet writes
Procedural Extractor	every turn	pattern detection, reinforcement or insertion
Episodic Decay	1.5 h	access-weighted decay, archive at 0.1, cold-store at 0.05
Codex Decay	1.5 h	edge strength decay, demote at strength < 0.3
Procedural Decay	1.5 h	boolean deactivation after 180 d if < 3 reinforcements
Clustering	30 min	assign unassigned turns to clusters, merge similar
Reflection	2 h	session synthesis, pattern crystallisation, slot proposals
Batch Summariser	2 h	50-turn batches, preservation-prompt summaries
Sentinel Monitor	30 min	declarative rules: threshold, absence, etc.
Fine-Tune	weekly (Mon 04:00 UTC)	retrain classifier head on curated labels
Compaction	manual	snapshot entities with ≥ 100 uncompact events
Drop Zone	filesystem	ingest .txt/.jsonl/.md into RAG store
Codex Inject Watcher	filesystem	ingest .yaml/.json into Codex with manual confidence

idempotency architecture, and operational infrastructure, is available in the technical report archived in the repository (`docs/ICE_Architecture[real_v2].md`).